

Group 10

Nano Processor

Project Report

Contributors

K.M.H.B.M. Udurawana – 230653J

N.Y.A. Jayasinghe – 230283A

U.G.I. Ranhiru – 230531J

U.L.K.S. Jayasinghe – 230286K

26. 03. 2025

Table of Contents

1.0 Basic Nano Processor

- 1.1 – Introduction
- 1.2 – Components
- 1.3 – Methodology
- 1.4 – Testing Methodology
- 1.5 – Troubleshooting and Design Refinement
- 1.6 – VHDL Codes (Design Source Files) & Design Schematic
 - 1.6.1 – Computer
 - 1.6.2 – Slow Clock
 - 1.6.3 – Nano Processor
 - 1.6.4 – Program Counter
 - 1.6.5 – Pc Incrementor
 - 1.6.6 – Next Address Selector
 - 1.6.7 – Instruction Decoder
 - 1.6.8 – Load Selector
 - 1.6.9 – Register Bank
 - 1.6.10 – 3 to 8 Decoder
 - 1.6.11 – 4-bit Register
 - 1.6.12 – Multiplexer
 - 1.6.13 – Arithmetic Unit
 - 1.6.14 – Ripple Carry Adder
 - 1.6.15 – Full Adder
 - 1.6.16 – Half Adder
 - 1.6.17 – Program ROM
 - 1.6.18 – Display Driver
- 1.7 – Simulation Files & Timing Diagrams
 - 1.7.1 – Computer
 - 1.7.2 – Program Counter
 - 1.7.3 – 3 to 8 Decoder
 - 1.7.4 – Arithmetic Unit
 - 1.7.5 – Instruction Decoder
 - 1.7.6 – Register Bank

1.8 – Constraints File

2.0 Optimized Nano Processor

2.1 – Objective

2.2 – Methodology

3.0 Extended Nano Processor

3.1 – Objective

3.2 – Methodology

3.3 – Extended Instruction Decoder

3.3.1–Source Code

3.3.2–Design Schematic

3.4 – Extended Arithmetic Unit

3.4.1–Source Code

3.4.2–Design Schematic

3.5 – Extended Program ROM

3.5.1–Source Code

3.5.2–Design Schematic

3.6 – Display Driver 1

3.6.1–Source Code

3.6.2–Design Schematic

3.7 – Display Driver 2

3.7.1–Source Code

Section 1

Basic Nano Processor

Introduction

In this project, we were assigned to design a nano processor utilizing our knowledge of designing previous components like adders, multiplexers, Decoders, Registers etc. Our initial assignment was the development of a basic nano processor from the reference schematic given in the assignment. We then optimized the design with the objective of minimizing the number of Look-Up Tables (LUTs) used, in accordance with a competitive contest. Then, we improved the optimized design by incorporating several features to increase its functionality. This report is organized into three main sections, each of which describes the basic, optimized, and feature-rich designs of the nano processor. We were supposed to design a 4-bit processor (nano processor) which has the ability of executing 4 instructions such as,

	Opcode		Operand									
MOV	1	0	R _a	R _a	R _a	X	X	X	d	d	d	d
ADD	0	0	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x
NEG	0	1	R _b	R _b	R _b	X	X	X	x	x	x	X
JZR	1	1	R _a	R _a	R _a	X	X	X	X	J	J	J

Fig 1.1.1: Instruction Set

Components

- **4-bit Adder/Subtractor Unit:** Performs addition or subtraction on two 4-bit numbers based on the mode control signal.
- **Program Counter:** Keeps track of the currently executing instruction by holding the address of the next instruction to be fetched.
- **3-bit Adder:** Increments the program counter by 1 to point to the next instruction.
- **Instruction Decoder:** Decodes the 12-bit instruction and generates the necessary control signals for other components.
- **Program ROM:** Stores the set of instructions to be executed by the processor.
- **2-way 3-bit Multiplexer:** Selects one of two 3-bit input values and outputs a 3-bit result based on a control signal.
- **2-way 4-bit Multiplexer:** Selects one of two 4-bit input values and outputs a 4-bit result based on a control signal.
- **8-way 4-bit Multiplexer:** Selects one of eight 4-bit input values and outputs a 4-bit result based on a 3-bit select signal.
- **Register Bank:** Provides temporary storage for data, allowing efficient access and manipulation during instruction execution.

After extending all the components we had to connect them accordingly using 3, 4 and 12-bit buses.

Methodology

In order to design the nano processor within a limited time frame, we divided the project among the team members based on each one's expertise and interest. Each of the team members worked on separate blocks, such as the ALU, control unit, register file, and memory interface. The block-based design allowed us to work in parallel, hence leading to faster development and effortless integration of each block with the entire processor architecture.

Following the development of the first version of the nano processor, we made a thorough study using the internet and consultations with senior students. The prime concern at this stage was minimizing the usage of Look-Up Tables (LUTs) and other FPGA resources while optimizing for maximum performance and additional features.

Following a round of review meetings, we decided to minimize Look-Up Tables using tactical logics and components from the IP catalog. Through these efforts, an improved version of our nano processor was created—marked by reduced LUT usage and additional features.

Work Division

Bimsara - spent approximately 24 hours working on wiring the nano processor and implementing key modules including the instruction decoder, register bank, and 4-bit register.

Sheron - contributed around 20 hours, focusing on the program counter, PC incrementor, program ROM, and display driver.

Yohan - dedicated about 18 hours, developing the slow clock, 3 to 8 decoder, arithmetic unit, ripple carry adder, full adder, and half adder.

Ishakya - worked for roughly 18 hours, completing the next address selector, load selector, and multiplexer.

After completing the basic version, all four team members collaborated closely to optimize the design by reducing LUT usage and to extend the nano processor with additional features. This final phase involved group discussions, debugging sessions, and extensive research to improve functionality and performance.

Testing Methodology

Since the design task was divided among members, each component was first designed and tested independently in the first stage. We verified the behavior of each module using schematic depictions and timing diagrams generated for specific sets of inputs.

Having verified all the modules, we integrated them to form the complete nano processor design. The integrated system was then thoroughly tested with both schematic-level simulation and timing analysis for verifying proper communication and synchronization between components.

To execute the final check, the highest-level design was synthesized and uploaded onto the FPGA Basys 3 board. This enabled real-time testing and confirmation of the operation of the processor in a physical environment, thereby guaranteeing that the system fulfilled functional as well as timing specifications under realistic hardware limitations.

Troubleshooting and Design Refinement

The first problem we encountered was a lack of understanding among group members regarding the different components and functions of the nano processor. This gap in knowledge made it challenging to effectively divide tasks and troubleshoot issues. To address this, we initiated collaborative knowledge-sharing sessions where each member contributed what they already knew. We also referred to our previous lab exercises, lecture notes, and lab reports to reinforce our understanding. This collective effort helped us build a stronger foundational grasp of the nano processor and allowed the group to move forward more confidently with the project.

During the design process, since each module was developed independently by different team members, we initially faced synchronization issues during integration. One of the primary challenges was the inconsistency in variable naming across modules, which made interconnecting components and ensuring correct signal flow difficult. To resolve this, we established a standardized naming convention and finalized a common set of variable names prior to integration. Additionally, we shared the implementation methods and logics of implementations of every module to the team members in order to get a well understanding of the current progress of the project

Another significant challenge arose during the optimization phase, specifically in our attempts to reduce Look-Up Table (LUT) usage. Not all approaches yielded positive results, some failed to reduce LUT utilization, while others inadvertently affected the processor's functionality or timing. To address this, we adopted a methodical approach: we thoroughly compared different implementations through simulation and synthesis

reports, researched best practices and case studies online, and analyzed the trade-offs associated with each method. This thorough analysis enabled us to determine the most efficient and stable optimizations that did not undermine the accuracy and performance of the nano processor.

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Bonded IOB (106)	BUFGCTRL (32)
Computer	25	42	14	25	7	15	1
NP1 (NanoProcessor)	24	15	7	24	6	0	0
SlowClk (Slow_Clk)	1	27	7	1	1	0	0

Fig 1.5.1: Utilization Report for the Basic Nano Processor

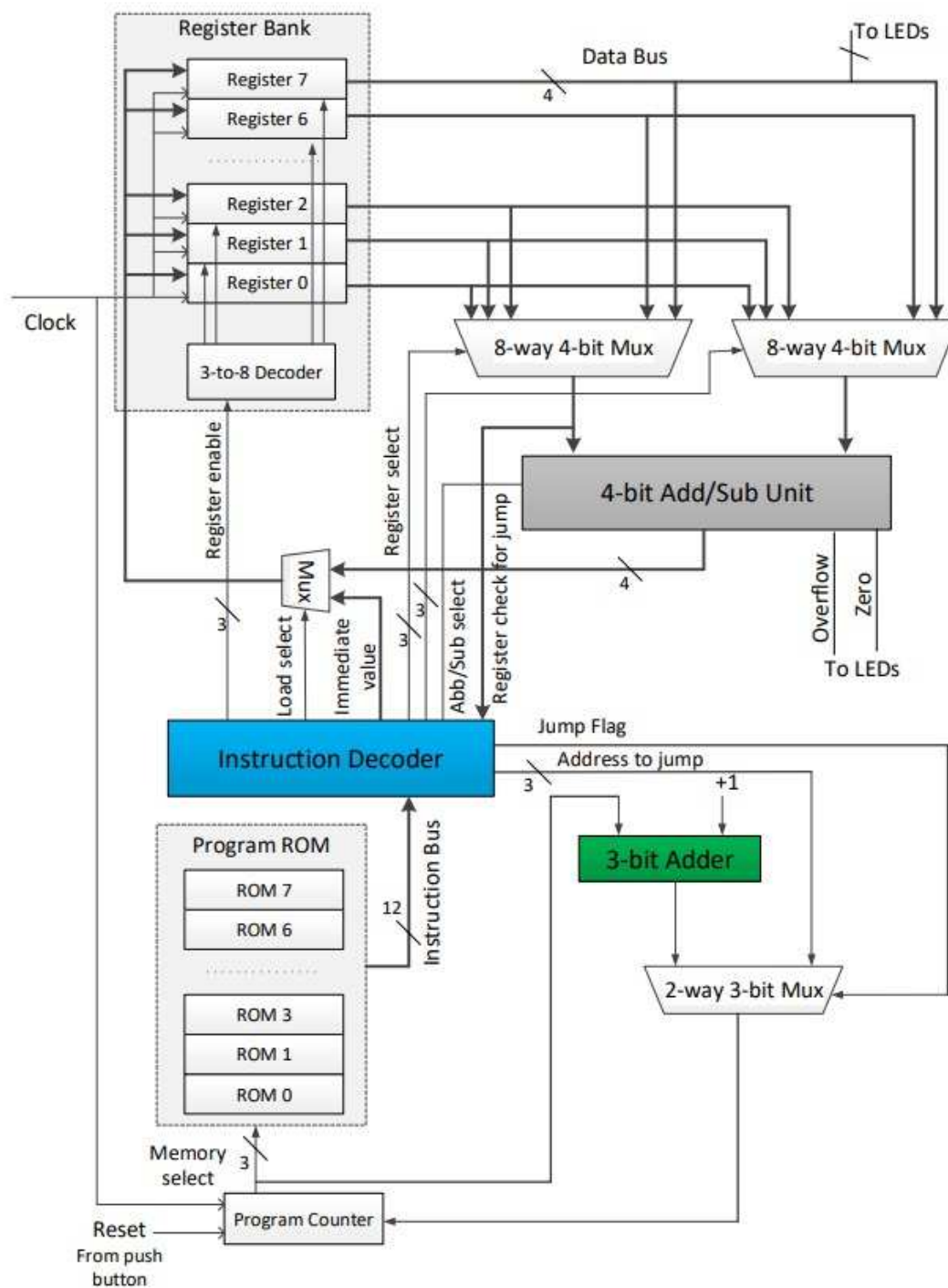


Fig 1.5.2: Abstraction of the Basic Nano Processor

Design Source Files & Design Schematics

Computer

Computer.vhd

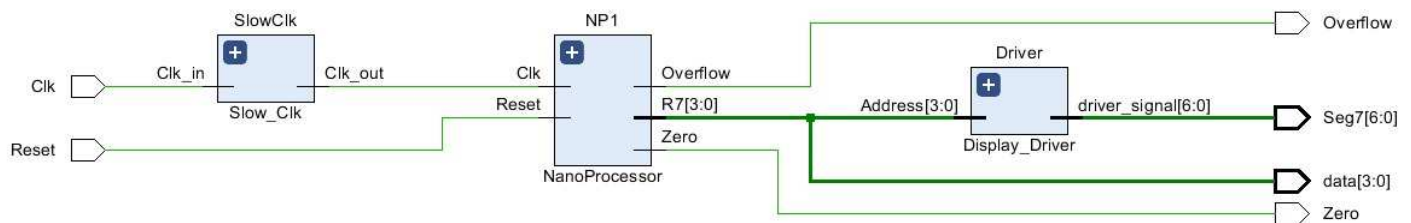
```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Computer is
5.     Port ( Clk : in STD_LOGIC;
6.           Reset : in STD_LOGIC;
7.           Seg7 : out STD_LOGIC_VECTOR (6 downto 0);
8.           data : out STD_LOGIC_VECTOR (3 downto 0);
9.           Zero : out STD_LOGIC;
10.          Overflow : out STD_LOGIC);
11. end Computer;
12.
13. architecture Behavioral of Computer is
14.
15.     Component Slow_Clk is
16.         generic (
17.             N : INTEGER := 26
18.         );
19.         Port (
20.             Clk_in : in STD_LOGIC;
21.             Clk_out : out STD_LOGIC
22.         );
23.     end Component;
24.
25.     Component Display_Driver is
26.         Port ( Address : in STD_LOGIC_VECTOR (3 downto 0);
27.             driver_signal : out STD_LOGIC_VECTOR (6 downto 0)
28.         );
29.     end Component;
30.
31.
32.     Component NanoProcessor is
33.         Port ( Reset : in STD_LOGIC;
34.             Clk : in STD_LOGIC;
35.             Overflow : out std_logic;
36.             Zero : out std_logic;
37.
38.             R7 : out STD_LOGIC_VECTOR (3 downto 0)
39.         );
40.     end Component;
41.
42.     Signal R7 : STD_LOGIC_VECTOR (3 downto 0);
43.     Signal SC : STD_LOGIC;
44.
45.
46. begin
47.
48.     SlowClk : Slow_Clk
49.         generic map( N => 26 )
```

```

50.      port map(
51.          Clk_in  => Clk,
52.          Clk_out => SC
53.      );
54.
55.
56. NP1 : NanoProcessor
57.     Port Map(
58.         Reset    => Reset,
59.         Clk       => SC,
60.         Overflow  => Overflow,
61.         Zero      => Zero,
62.         R7        => R7
63.     );
64.
65. Driver : Display_Driver
66.     Port Map(
67.         Address    => R7,
68.         driver_signal => Seg7
69.     );
70.
71. data <= R7;
72. end Behavioral;
73.

```

Design Schematic – Computer

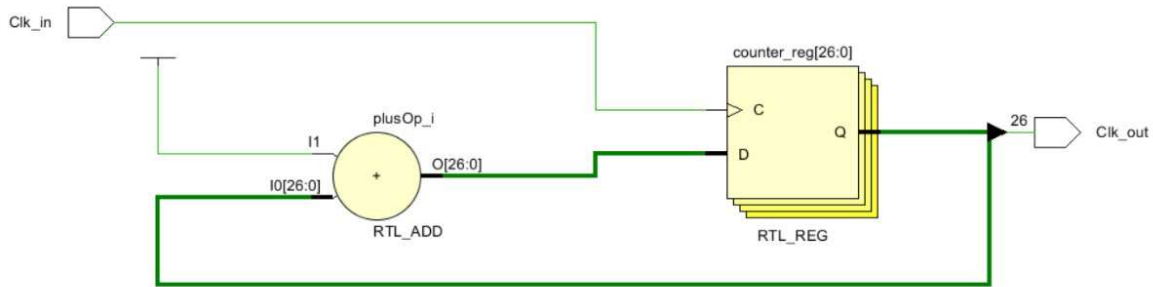


Slow Clock

Slow_Clk.vhd

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity Slow_Clk is
6.     generic (
7.         N : INTEGER := 26
8.     );
9.     Port (
10.         Clk_in  : in  STD_LOGIC;
11.         Clk_out : out STD_LOGIC
12.     );
13. end Slow_Clk;
14.
15. architecture Behavioral of Slow_Clk is
16.
17.     signal counter : unsigned(26 downto 0) := (others => '0'); -- 27-bit binary
counter
18.
19. begin
20.     process(Clk_in)
21.     begin
22.         if rising_edge(Clk_in) then
23.             counter <= counter + 1;
24.         end if;
25.     end process;
26.
27.     Clk_out <= counter(N);
28.
29. end Behavioral;
```

Design Schematic – Slow Clock



Nano Processor

NanoProcessor.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity NanoProcessor is
5.     Port ( Reset      : in STD_LOGIC;
6.           Clk         : in STD_LOGIC;
7.           Overflow    : out std_logic;
8.           Zero        : out std_logic;
9.
10.          R7 : out STD_LOGIC_VECTOR (3 downto 0)
11.          );
12. end NanoProcessor;
13.
14. architecture Behavioral of NanoProcessor is
15.
16.     Component Instruction_Decoder is
17.         Port (
18.             Instruction : in  STD_LOGIC_VECTOR(11 downto 0);
19.             JumpCheck   : in  STD_LOGIC_VECTOR(3  downto 0);
20.             RegEn       : out STD_LOGIC_VECTOR(2  downto 0);
21.             Sel_A       : out STD_LOGIC_VECTOR(2  downto 0);
22.             Sel_B       : out STD_LOGIC_VECTOR(2  downto 0);
23.             LoadSel     : out STD_LOGIC;
24.             ImmVal      : out STD_LOGIC_VECTOR(3  downto 0);
25.             Sub         : out STD_LOGIC;

```

```

26.         JumpFlag      : out STD_LOGIC;
27.         JumpAddress    : out STD_LOGIC_VECTOR(2 downto 0)
28.     );
29. end Component;
30.
31. Component MUX8 is
32.     Port (
33.         Sel  : in  STD_LOGIC_VECTOR(2 downto 0);
34.         I0   : in  STD_LOGIC_VECTOR(3 downto 0);
35.         I1   : in  STD_LOGIC_VECTOR(3 downto 0);
36.         I2   : in  STD_LOGIC_VECTOR(3 downto 0);
37.         I3   : in  STD_LOGIC_VECTOR(3 downto 0);
38.         I4   : in  STD_LOGIC_VECTOR(3 downto 0);
39.         I5   : in  STD_LOGIC_VECTOR(3 downto 0);
40.         I6   : in  STD_LOGIC_VECTOR(3 downto 0);
41.         I7   : in  STD_LOGIC_VECTOR(3 downto 0);
42.         O     : out STD_LOGIC_VECTOR(3 downto 0)
43.     );
44. end Component;
45.
46. Component MUX2 is
47.     port(
48.         Sel : in std_logic;
49.         A   : in std_logic_vector(2 downto 0);
50.         B   : in std_logic_vector(2 downto 0);
51.         O   : out std_logic_vector(2 downto 0)
52.     );
53. end Component;
54.
55. Component Load_Selector is
56.     Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
57.           B : in STD_LOGIC_VECTOR (3 downto 0);
58.           Sel : in STD_LOGIC;
59.           O : out STD_LOGIC_VECTOR (3 downto 0));
60. end Component;
61.
62. Component PC_Inc is
63.     Port ( Q : in STD_LOGIC_VECTOR (2 downto 0);
64.           D : out STD_LOGIC_VECTOR (2 downto 0));
65. end Component;
66.
67. Component Program_ROM is
68.     port(ROM_address : in STD_LOGIC_VECTOR (2 downto 0);
69.          I: out STD_LOGIC_VECTOR (11 downto 0)
70.     );
71. end Component;
72.
73. Component Register_PC is
74.     Port (
75.         Clk      : in STD_LOGIC;
76.         Res      : in STD_LOGIC;
77.         D        : in STD_LOGIC_VECTOR(2 downto 0);
78.         Q        : out STD_LOGIC_VECTOR(2 downto 0)
79.     );
80. end Component;
81.
82. Component AU is
83.     Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
84.           B : in STD_LOGIC_VECTOR (3 downto 0);
85.           Sub : in STD_LOGIC;
86.           S : out STD_LOGIC_VECTOR (3 downto 0);
87.           Overflow : out STD_LOGIC;
88.           Zero : out STD_LOGIC);
89. end Component;
90.

```

```

91. Component Register_Bank is
92. PORT (
93.     RegEn    : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
94.     Data     : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
95.     Clk      : IN  STD_LOGIC;
96.     Res      : IN  STD_LOGIC;
97.     Reg0     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
98.     Reg1     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
99.     Reg2     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
100.    Reg3     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
101.    Reg4     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
102.    Reg5     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
103.    Reg6     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
104.    Reg7     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
105. );
106. END Component;
107.
108. -----
109. -----
110.
111.     signal Next_Address      : STD_LOGIC_VECTOR(2 DOWNTO 0);
112.     signal Current_Address   : STD_LOGIC_VECTOR(2 DOWNTO 0);
113.     signal Selected_Address  : STD_LOGIC_VECTOR(2 DOWNTO 0);
114.     signal Jump_Address      : STD_LOGIC_VECTOR(2 DOWNTO 0);
115.     signal Jump_Flag         : std_logic;
116.     signal Instruction        : STD_LOGIC_VECTOR(11 DOWNTO 0);
117.
118.     signal t0, t1, t2, t3, t4, t5, t6, t7 : STD_LOGIC_VECTOR(3 DOWNTO 0);
119.
120.     signal Load_Selection    : std_logic;
121.     signal Immediate_Value   : STD_LOGIC_VECTOR(3 DOWNTO 0);
122.     signal ASelect           : STD_LOGIC_VECTOR(2 DOWNTO 0);
123.     signal BSelect           : STD_LOGIC_VECTOR(2 DOWNTO 0);
124.     signal AData             : STD_LOGIC_VECTOR(3 DOWNTO 0);
125.     signal BData             : STD_LOGIC_VECTOR(3 DOWNTO 0);
126.     signal Operation_Result  : STD_LOGIC_VECTOR(3 DOWNTO 0);
127.     signal Sub_Sel           : std_logic;
128.     signal Register_Enable   : STD_LOGIC_VECTOR(2 DOWNTO 0);
129.     signal Selected_Load     : STD_LOGIC_VECTOR(3 DOWNTO 0);
130.
131. -----
132. -----
133.
134.
135. begin
136.
137. Program_Counter : Register_PC
138. Port Map (
139.     Clk => Clk,
140.     Res => Reset,
141.     D => Selected_Address,
142.     Q => Current_Address
143. );
144.
145. Incrementer : PC_Inc
146. Port Map (
147.     Q => Current_Address,
148.     D => Next_Address
149. );
150.
151. Next_Address_Selector : MUX2
152. Port Map (
153.     Sel => Jump_Flag,
154.     A => Next_Address,
155.     B => Jump_Address,

```



```

156.         0 => Selected_Address
157.     );
158.
159. InstructionDecoder : Instruction_Decoder
160.     Port Map (
161.         Instruction => Instruction,
162.         JumpCheck  => AData,
163.         RegEn      => Register_Enable,
164.         Sel_A      => ASelect,
165.         Sel_B      => BSelect,
166.         LoadSel    => Load_Selection,
167.         ImmVal     => Immediate_Value,
168.         Sub        => Sub_Sel,
169.         JumpFlag   => Jump_Flag,
170.         JumpAddress => Jump_Address
171.     );
172.
173. LoadSelector : Load_Selector
174.     Port Map (
175.         A  => Operation_Result,
176.         B  => Immediate_Value,
177.         Sel => Load_Selection,
178.         0  => Selected_Load
179.     );
180.
181.
182. RegisterBank : Register_Bank
183.     Port Map (
184.         RegEn  => Register_Enable,
185.         Data   => Selected_Load,
186.         Clk    => Clk,
187.         Res    => Reset,
188.         Reg0   => t0,
189.         Reg1   => t1,
190.         Reg2   => t2,
191.         Reg3   => t3,
192.         Reg4   => t4,
193.         Reg5   => t5,
194.         Reg6   => t6,
195.         Reg7   => t7
196.     );
197.
198. Selector_A : MUX8
199.     Port Map (
200.         Sel  => ASelect,
201.         I0   => t0,
202.         I1   => t1,
203.         I2   => t2,
204.         I3   => t3,
205.         I4   => t4,
206.         I5   => t5,
207.         I6   => t6,
208.         I7   => t7,
209.         0    => AData
210.     );
211.
212. Selector_B : MUX8
213.     Port Map (
214.         Sel  => BSelect,
215.         I0   => t0,
216.         I1   => t1,
217.         I2   => t2,
218.         I3   => t3,
219.         I4   => t4,
220.         I5   => t5,

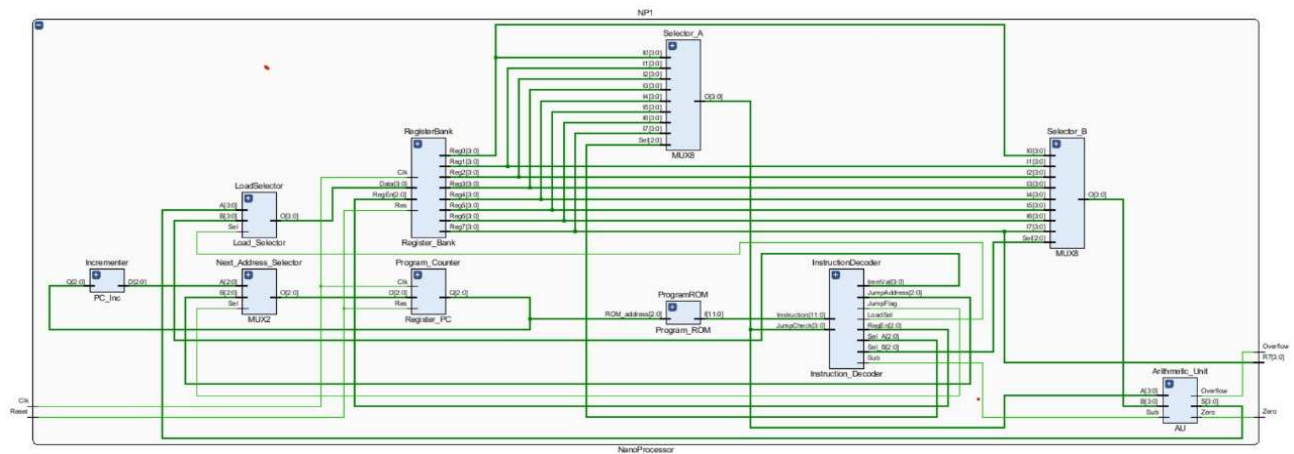
```

```

221.      I6    => t6,
222.      I7    => t7,
223.      0     => BData
224.    );
225.
226. Arithmetic_Unit : AU
227.   Port Map(
228.       A => AData,
229.       B => BData,
230.       Sub => Sub_Sel,
231.       S => Operation_Result,
232.       Overflow => Overflow,
233.       Zero => Zero
234.   );
235.
236. ProgramROM : Program_ROM
237.   Port Map(
238.       ROM_address => Current_Address,
239.       I => Instruction
240.   );
241.
242. R7 <= t7;
243.
244. end Behavioral;
245.

```

Design Schematic - Nano Processor



Program Counter

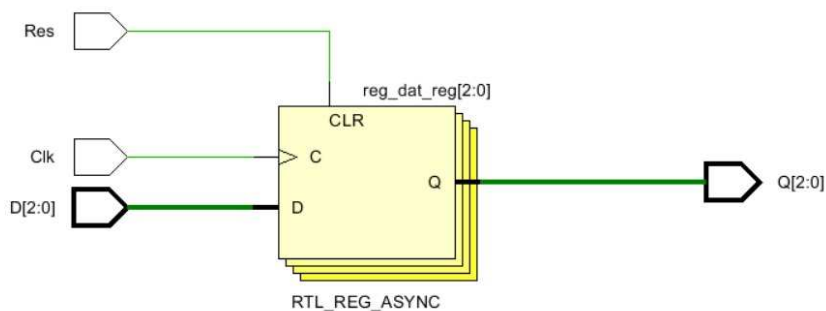
Register_PC.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Register_PC is
5.     Port (
6.         Clk : in  STD_LOGIC;           -- Clock input
7.         Res : in  STD_LOGIC;           -- Reset input (active high)
8.         D   : in  STD_LOGIC_VECTOR(2 downto 0); -- 3-bit data input
9.         Q   : out STD_LOGIC_VECTOR(2 downto 0) -- 3-bit data output
10.    );
11. end Register_PC;
12.
13. architecture Behavioral of Register_PC is
14.     signal reg_dat : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');
15. begin
16.
17.     process(Clk, Res)
18.     begin
19.         if Res = '1' then
20.             reg_dat <= (others => '0');
21.         elsif rising_edge(Clk) then
22.             reg_dat <= D;
23.         end if;
24.     end process;
25.
26.     Q <= reg_dat;
27.
28. end Behavioral;

```

Design Schematic – Program Counter



PC Incrementor

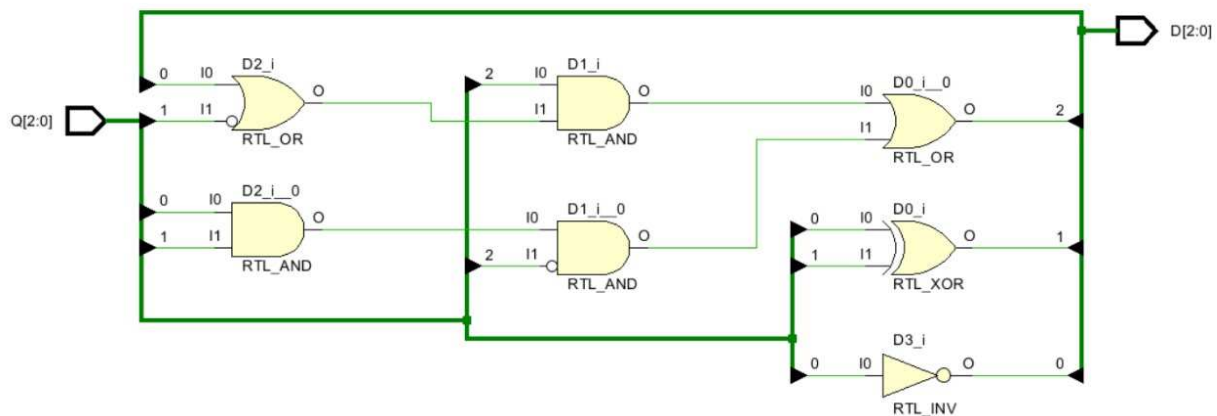
PC_Inc.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity PC_Inc is
5.     Port ( Q : in STD_LOGIC_VECTOR (2 downto 0);
6.           D : out STD_LOGIC_VECTOR (2 downto 0));
7. end PC_Inc;
8.
9. architecture Behavioral of PC_Inc is
10.
11. begin
12. D(0) <= NOT Q(0);
13. D(1) <= Q(0) XOR Q(1);
14. D(2) <= ((Q(2)) AND ((NOT Q(0)) OR (NOT Q(1)))) OR ((Q(0)) AND (Q(1)) AND (NOT
Q(2)));
15. --D(2) <= ((NOT Q(0)) AND Q(2)) OR (Q(2) AND (NOT Q(1))) OR (Q(0) AND Q(1) AND
(NOT Q(2)));
16.
17. end Behavioral;
18.

```

Design Schematic – PC Incrementor



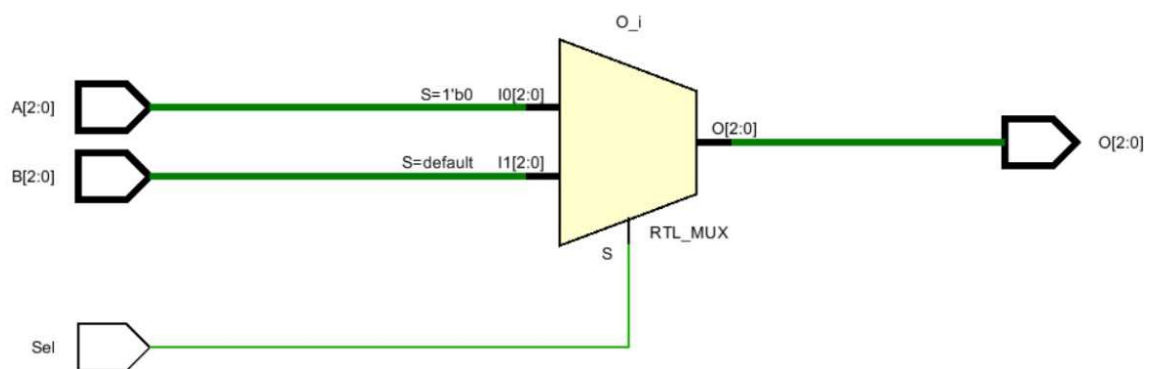
Next Address Selector (MUX2.vhd)

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity MUX2 is
5.     Port (
6.         Sel : in  std_logic;
7.         A   : in  std_logic_vector(2 downto 0);
8.         B   : in  std_logic_vector(2 downto 0);
9.         O   : out std_logic_vector(2 downto 0)
10.    );
11. end MUX2;
12.
13. architecture Behavioral of MUX2 is
14. begin
15.     process(Sel, A, B)
16.     begin
17.         if Sel = '0' then
18.             O <= A;
19.         else
20.             O <= B;
21.         end if;
22.     end process;
23. end Behavioral;
24.

```

Design Schematic – Next Address Selector



Instruction Decoder

Instruction Decoder.vhd

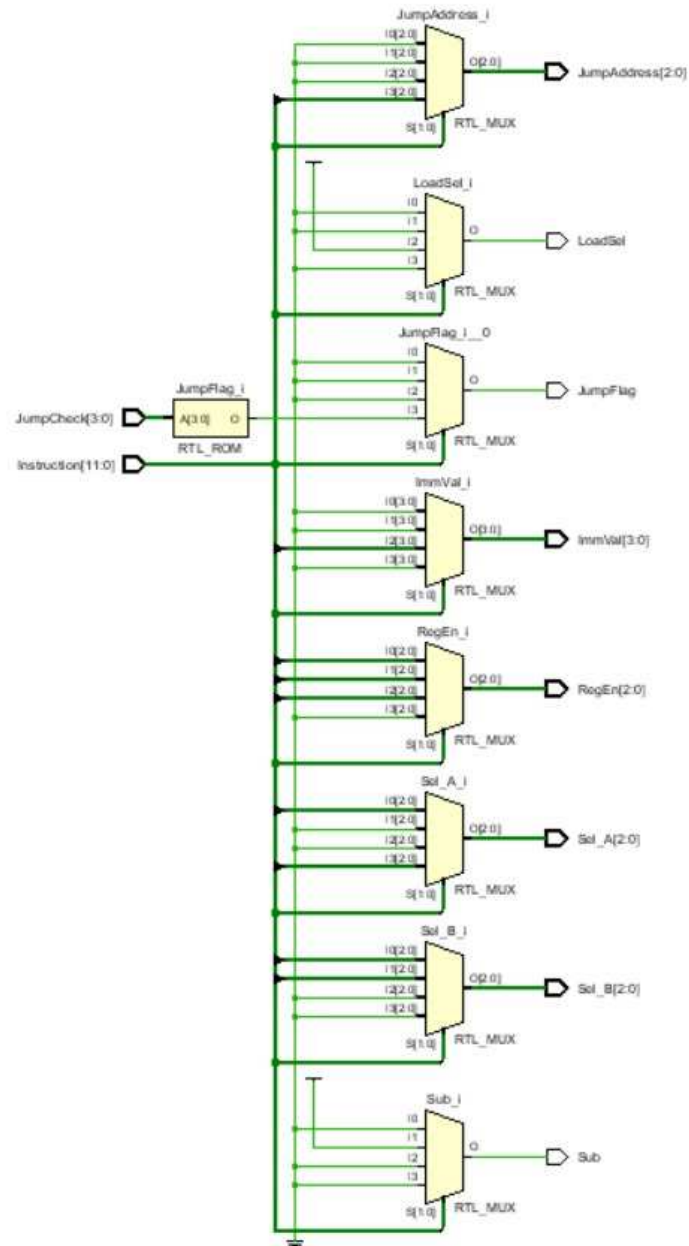
```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Instruction_Decoder is
5.     Port (
6.         Instruction    : in  STD_LOGIC_VECTOR(11 downto 0);
7.         JumpCheck      : in  STD_LOGIC_VECTOR(3 downto 0);
8.         RegEn          : out STD_LOGIC_VECTOR(2 downto 0);
9.         Sel_A          : out STD_LOGIC_VECTOR(2 downto 0);
10.        Sel_B          : out STD_LOGIC_VECTOR(2 downto 0);
11.        LoadSel        : out STD_LOGIC;
12.        ImmVal         : out STD_LOGIC_VECTOR(3 downto 0);
13.        Sub            : out STD_LOGIC;
14.        JumpFlag       : out STD_LOGIC;
15.        JumpAddress    : out STD_LOGIC_VECTOR(2 downto 0)
16.    );
17. end Instruction_Decoder;
18.
19. architecture Behavioral of Instruction_Decoder is
20.     signal opcode : STD_LOGIC_VECTOR(1 downto 0);
21. begin
22.
23.     opcode <= Instruction(11 downto 10);
24.
25.     process(opcode, Instruction, JumpCheck)
26.     begin
27.         -- Default outputs
28.         RegEn    <= (others => '0');
29.         Sel_A    <= (others => '0');
30.         Sel_B    <= (others => '0');
31.         LoadSel  <= '0';
32.         ImmVal   <= (others => '0');
33.         Sub      <= '0';
34.         JumpFlag <= '0';
35.         JumpAddress <= (others => '0');
36.
37.         case opcode is
38.             when "00" => -- ADD
39.                 RegEn <= Instruction(9 downto 7);
40.                 Sel_A <= Instruction(9 downto 7);
41.                 Sel_B <= Instruction(6 downto 4);
42.
43.             when "01" => -- NEG
44.                 RegEn <= Instruction(9 downto 7);
45.                 Sel_B <= Instruction(9 downto 7);

```

```
46.         Sub    <= '1';
47.
48.     when "10" => -- MOVI
49.         RegEn    <= Instruction(9 downto 7);
50.         LoadSel  <= '1';
51.         ImmVal    <= Instruction(3 downto 0);
52.
53.     when "11" => -- JZR
54.         Sel_A      <= Instruction(9 downto 7);
55.         JumpAddress <= Instruction(2 downto 0);
56.         if JumpCheck = "0000" then
57.             JumpFlag <= '1';
58.         end if;
59.
60.     when others =>
61.         -- Already covered by default assignments
62.         null;
63.     end case;
64. end process;
65. end Behavioral;
66.
```

Design Schematic – Instruction Decoder



Load Selector

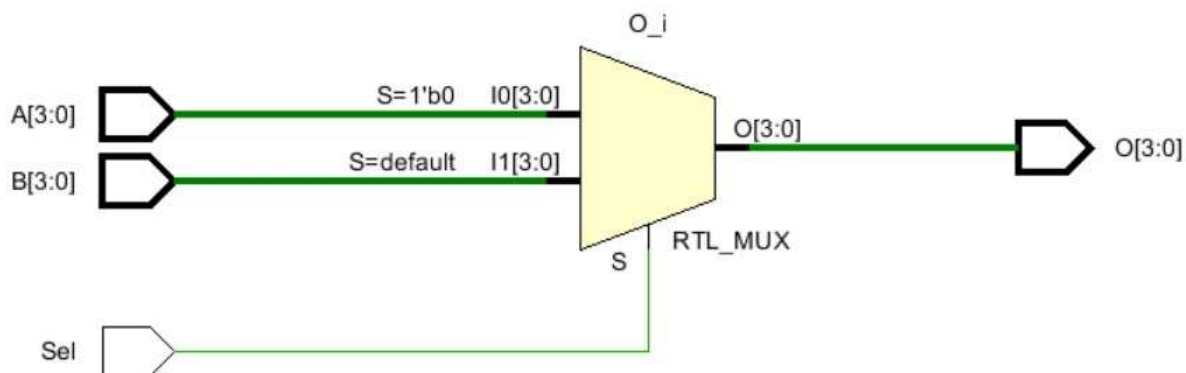
Load_Selector.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Load_Selector is
5.     Port (
6.         A      : in STD_LOGIC_VECTOR (3 downto 0);
7.         B      : in STD_LOGIC_VECTOR (3 downto 0);
8.         Sel     : in STD_LOGIC;
9.         O       : out STD_LOGIC_VECTOR (3 downto 0)
10.    );
11. end Load_Selector;
12.
13. architecture Behavioral of Load_Selector is
14. begin
15.     process(A, B, Sel)
16.     begin
17.         if Sel = '0' then
18.             O <= A;
19.         else
20.             O <= B;
21.         end if;
22.     end process;
23. end Behavioral;
24.

```

Design Schematic – Load Selector



Register Bank

Register_Bank.vhd

```

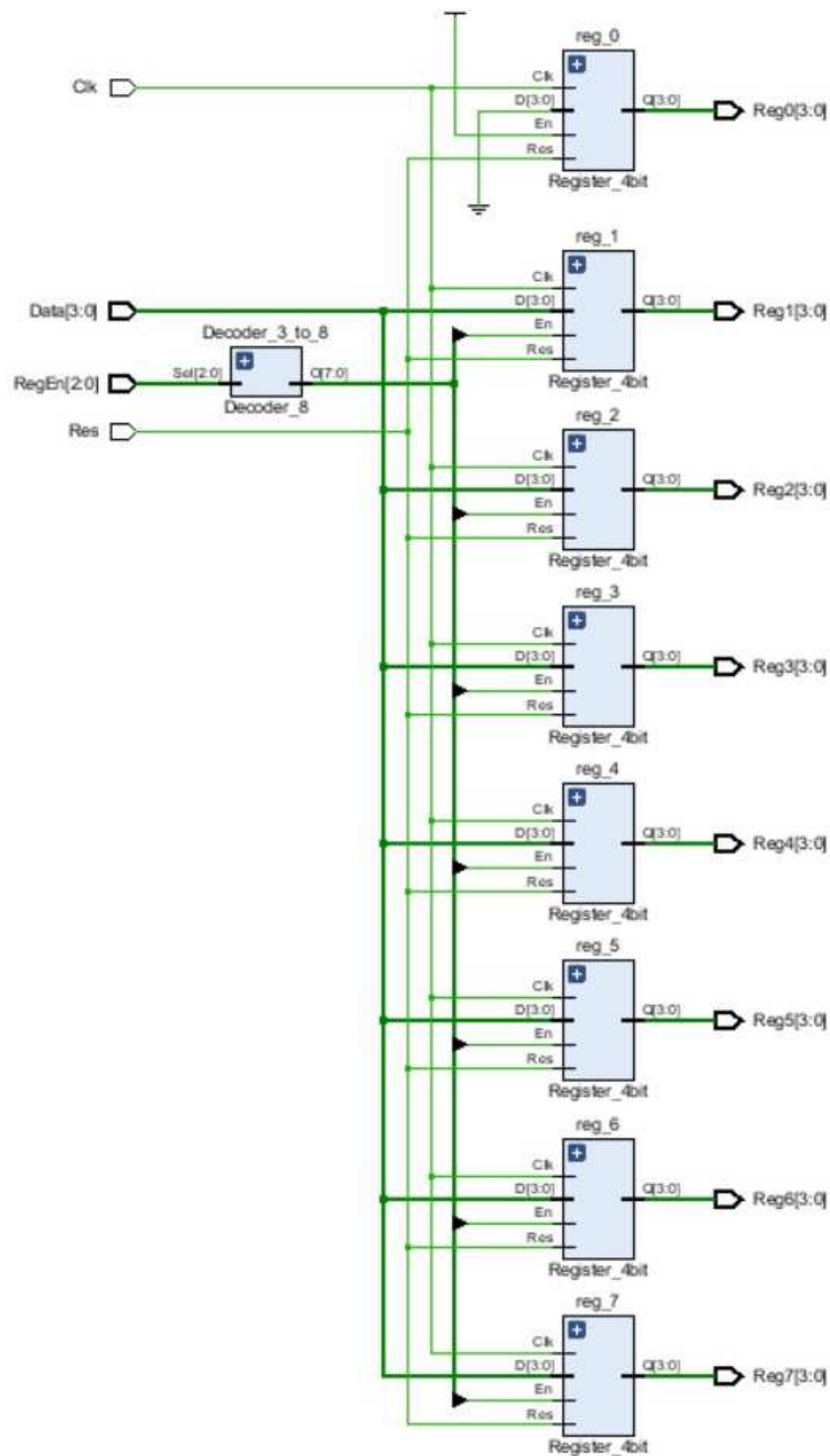
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Register_Bank is
5. PORT (
6.     RegEn    : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
7.     Data     : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
8.     Clk      : IN  STD_LOGIC;
9.     Res      : IN  STD_LOGIC;
10.    Reg0     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
11.    Reg1     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
12.    Reg2     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
13.    Reg3     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
14.    Reg4     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
15.    Reg5     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
16.    Reg6     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
17.    Reg7     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
18. );
19. END Register_Bank;
20.
21. ARCHITECTURE Behavioral OF Register_Bank IS
22.
23.     COMPONENT Register_4bit
24.     PORT (
25.         Clk    : IN  STD_LOGIC;
26.         Res    : IN  STD_LOGIC;
27.         En     : IN  STD_LOGIC;
28.         D      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
29.         Q      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
30.     );
31.     END COMPONENT;
32.
33.     COMPONENT Decoder_8
34.     PORT (
35.         Sel : in  STD_LOGIC_VECTOR(2 downto 0);
36.         O   : out STD_LOGIC_VECTOR(7 downto 0);
37.     );
38.     END COMPONENT;
39.
40.     SIGNAL Reg_Sel : STD_LOGIC_VECTOR(7 DOWNTO 0);
41.
42. BEGIN
43.
44.     Decoder_3_to_8 : Decoder_8
45.     PORT MAP (
46.         Sel => RegEn,
47.         O   => Reg_Sel
48.     );

```

```
49.
50.     reg_0 : Register_4bit
51.         PORT MAP (
52.             Clk  => Clk,
53.             Res  => Res,
54.             En   => '1',
55.             D    => "0000",
56.             Q    => Reg0
57.         );
58.
59.     reg_1 : Register_4bit
60.         PORT MAP (
61.             Clk  => Clk,
62.             Res  => Res,
63.             En   => Reg_Sel(1),
64.             D    => Data,
65.             Q    => Reg1
66.         );
67.
68.     reg_2 : Register_4bit
69.         PORT MAP (
70.             Clk  => Clk,
71.             Res  => Res,
72.             En   => Reg_Sel(2),
73.             D    => Data,
74.             Q    => Reg2
75.         );
76.
77.     reg_3 : Register_4bit
78.         PORT MAP (
79.             Clk  => Clk,
80.             Res  => Res,
81.             En   => Reg_Sel(3),
82.             D    => Data,
83.             Q    => Reg3
84.         );
85.
86.     reg_4 : Register_4bit
87.         PORT MAP (
88.             Clk  => Clk,
89.             Res  => Res,
90.             En   => Reg_Sel(4),
91.             D    => Data,
92.             Q    => Reg4
93.         );
94.
95.     reg_5 : Register_4bit
96.         PORT MAP (
97.             Clk  => Clk,
98.             Res  => Res,
99.             En   => Reg_Sel(5),
100.            D    => Data,
101.            Q    => Reg5
102.        );
103.
```

```
104.     reg_6 : Register_4bit
105.         PORT MAP (
106.             Clk  => Clk,
107.             Res  => Res,
108.             En   => Reg_Sel(6),
109.             D    => Data,
110.             Q    => Reg6
111.         );
112.
113.     reg_7 : Register_4bit
114.         PORT MAP (
115.             Clk  => Clk,
116.             Res  => Res,
117.             En   => Reg_Sel(7),
118.             D    => Data,
119.             Q    => Reg7
120.         );
121.
122. END Behavioral;
123.
124.
```

Design Schematic – Register Bank



3 to 8 Decoder

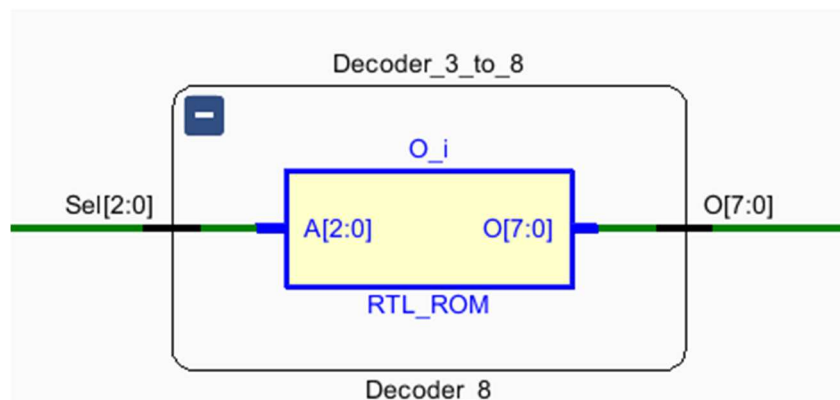
Decoder_3_to_8.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_VECTOR(2 downto 0);
3.
4. entity Decoder_8 is
5.     Port (
6.         Sel : in  STD_LOGIC_VECTOR(2 downto 0);
7.         O   : out STD_LOGIC_VECTOR(7 downto 0)
8.     );
9. end Decoder_8;
10.
11. architecture Behavioral of Decoder_8 is
12. begin
13.     process(Sel)
14.     begin
15.         case Sel is
16.             when "000" => O <= "00000001";
17.             when "001" => O <= "00000010";
18.             when "010" => O <= "00000100";
19.             when "011" => O <= "00001000";
20.             when "100" => O <= "00010000";
21.             when "101" => O <= "00100000";
22.             when "110" => O <= "01000000";
23.             when "111" => O <= "10000000";
24.             when others => O <= (others => '0');
25.         end case;
26.     end process;
27. end Behavioral;
28.

```

Design Schematic – 3 to 8 Decoder

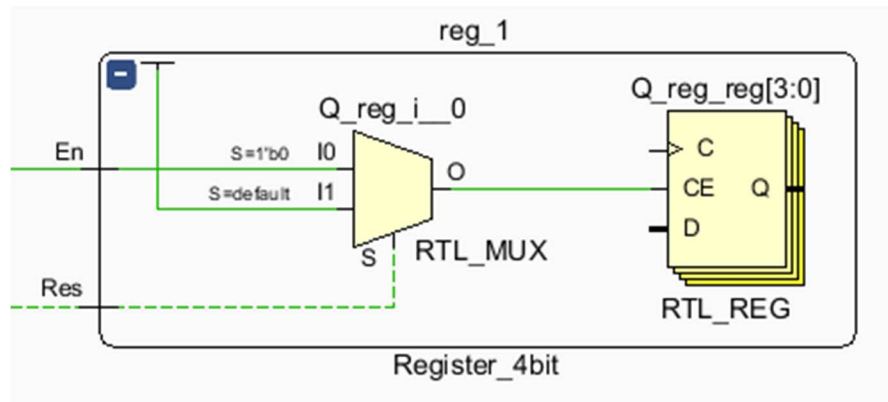


4-bit Register

(Register_4bit.vhd)

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Register_4bit is
5.     Port (
6.         Clk      : in STD_LOGIC;
7.         Res      : in STD_LOGIC;
8.         En       : in STD_LOGIC;
9.         D        : in STD_LOGIC_VECTOR(3 downto 0);
10.        Q        : out STD_LOGIC_VECTOR(3 downto 0)
11.    );
12. end Register_4bit;
13.
14. architecture Behavioral of Register_4bit is
15.     signal Q_reg : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
16. begin
17.     process(Clk)
18.     begin
19.         if rising_edge(Clk) then
20.             if Res = '0' then
21.                 if En = '1' then
22.                     Q_reg <= D;
23.                 end if;
24.             else
25.                 Q_reg <= "0000";
26.             end if;
27.         end if;
28.     end process;
29.
30.     Q <= Q_reg;
31. end Behavioral;
32.
```

Design Schematic – 3 to 8 Decoder



Multiplexer

MUX8.vhd

```

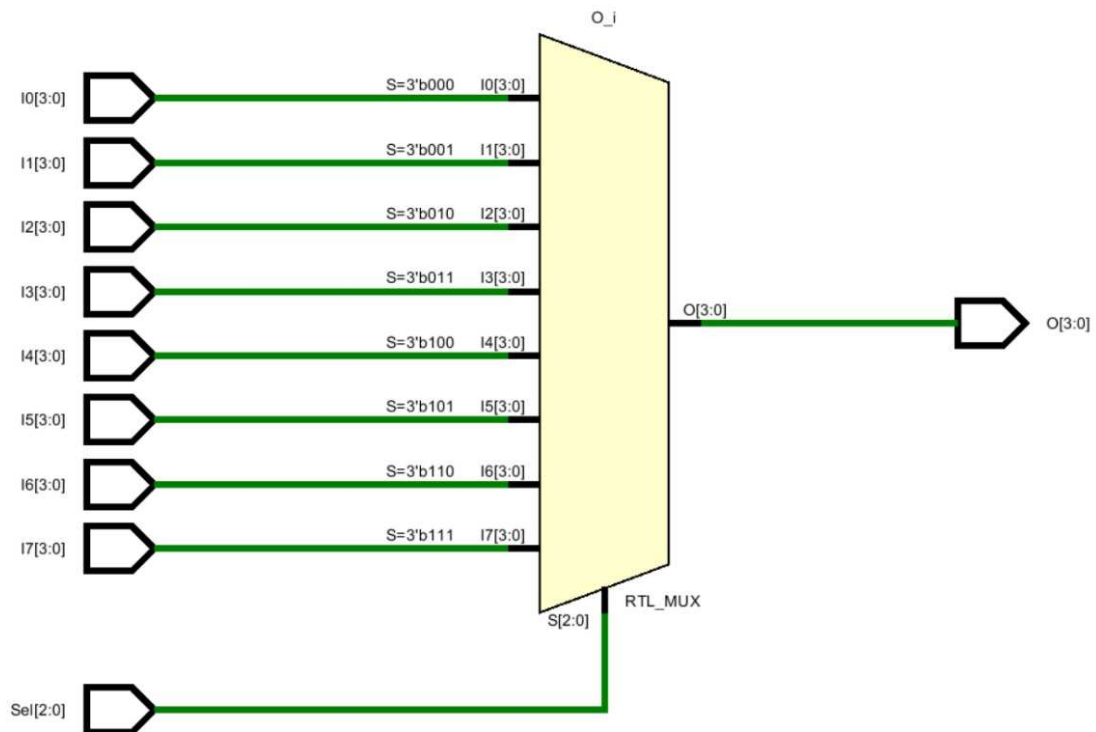
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity MUX8 is
5.     Port (
6.         Sel  : in  STD_LOGIC_VECTOR(2 downto 0);
7.         I0   : in  STD_LOGIC_VECTOR(3 downto 0);
8.         I1   : in  STD_LOGIC_VECTOR(3 downto 0);
9.         I2   : in  STD_LOGIC_VECTOR(3 downto 0);
10.        I3   : in  STD_LOGIC_VECTOR(3 downto 0);
11.        I4   : in  STD_LOGIC_VECTOR(3 downto 0);
12.        I5   : in  STD_LOGIC_VECTOR(3 downto 0);
13.        I6   : in  STD_LOGIC_VECTOR(3 downto 0);
14.        I7   : in  STD_LOGIC_VECTOR(3 downto 0);
15.        O    : out STD_LOGIC_VECTOR(3 downto 0)
16.    );
17. end MUX8;
18.
19. architecture Behavioral of MUX8 is
20. begin
21.     process(Sel, I0, I1, I2, I3, I4, I5, I6, I7)
22.     begin
23.         case Sel is
24.             when "000" => O <= I0;
25.             when "001" => O <= I1;
26.             when "010" => O <= I2;

```



```
27.         when "011" => O <= I3;  
28.         when "100" => O <= I4;  
29.         when "101" => O <= I5;  
30.         when "110" => O <= I6;  
31.         when "111" => O <= I7;  
32.         when others => O <= (others => '0');  
33.     end case;  
34. end process;  
35. end Behavioral;  
36.
```

Design Schematic – Multiplexer

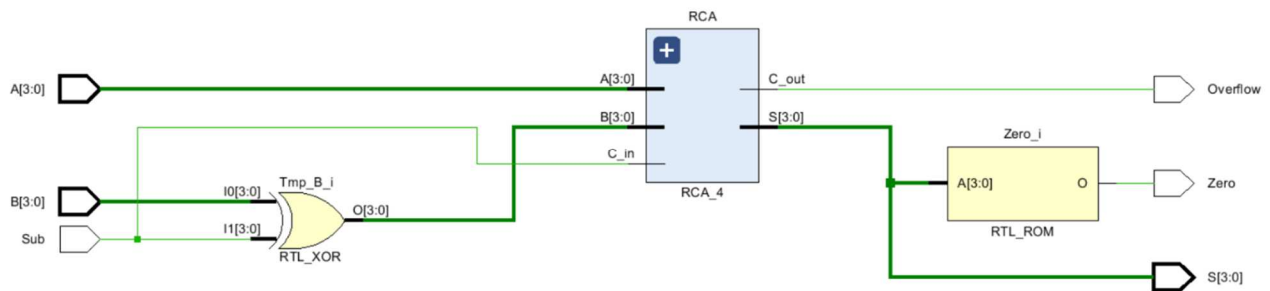


Arithmetic Unit

AU.vhd

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity AU is
5.     Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
6.           B : in STD_LOGIC_VECTOR (3 downto 0);
7.           Sub : in STD_LOGIC;
8.           S : out STD_LOGIC_VECTOR (3 downto 0);
9.           Overflow : out STD_LOGIC;
10.          Zero : out STD_LOGIC);
11. end AU;
12.
13. architecture Behavioral of AU is
14.     Component RCA_4 is
15.         Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
16.               B : in STD_LOGIC_VECTOR (3 downto 0);
17.               C_in : in STD_LOGIC;
18.               S : out STD_LOGIC_VECTOR (3 downto 0);
19.               C_out : out STD_LOGIC);
20.     end Component;
21.
22.     Signal Tmp_B : STD_LOGIC_VECTOR (3 downto 0);
23.     Signal Sum : STD_LOGIC_VECTOR (3 downto 0);
24.
25.     begin
26.         RCA : RCA_4
27.         port map(
28.             A => A,
29.             B => Tmp_B,
30.             C_in => Sub,
31.             S => Sum,
32.             C_out => Overflow
33.         );
34.
35.         Tmp_B <= B XOR (Sub & Sub & Sub & Sub);
36.         S <= Sum;
37.
38.         Zero <= '1' when Sum = "0000" else '0';
39.
40.
41.     end Behavioral;
42.
```

Design Schematic – AU



Ripple Carry Adder

RCA.vhd

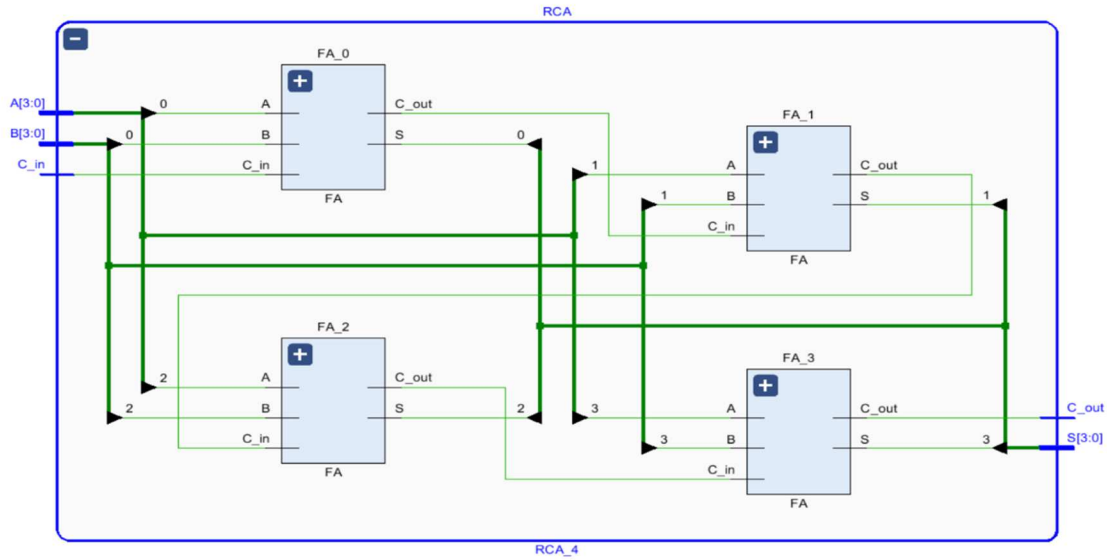
```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4.
5. entity RCA_4 is
6.     Port (      A : in STD_LOGIC_VECTOR (3 downto 0);
7.             B : in STD_LOGIC_VECTOR (3 downto 0);
8.             C_in : in STD_LOGIC;
9.             S : out STD_LOGIC_VECTOR (3 downto 0);
10.            C_out : out STD_LOGIC);
11. end RCA_4;
12.
13. architecture Behavioral of RCA_4 is
14. component FA
15. port (
16. A: in std_logic;
17. B: in std_logic;
18. C_in: in std_logic;
19. S: out std_logic;
20. C_out: out std_logic);
21. end component;
22.
23. SIGNAL FA0_S, FA0_C, FA1_S, FA1_C, FA2_S, FA2_C, FA3_S, FA3_C : std_logic;

```

```
24.  
25. begin  
26.  
27. FA_0 : FA  
28.   port map (  
29.     A => A(0),  
30.     B => B(0),  
31.     C_in => C_in,  
32.     S => S(0),  
33.     C_Out => FA0_C);  
34.  
35. FA_1 : FA  
36.   port map (  
37.     A => A(1),  
38.     B => B(1),  
39.     C_in => FA0_C,  
40.     S => S(1),  
41.     C_Out => FA1_C);  
42.  
43. FA_2 : FA  
44.   port map (  
45.     A => A(2),  
46.     B => B(2),  
47.     C_in => FA1_C,  
48.     S => S(2),  
49.     C_Out => FA2_C);  
50.  
51. FA_3 : FA  
52.   port map (  
53.     A => A(3),  
54.     B => B(3),  
55.     C_in => FA2_C,  
56.     S => S(3),  
57.     C_Out => C_out);  
58.  
59.  
60. end Behavioral;  
61.
```

Design Schematic – RCA

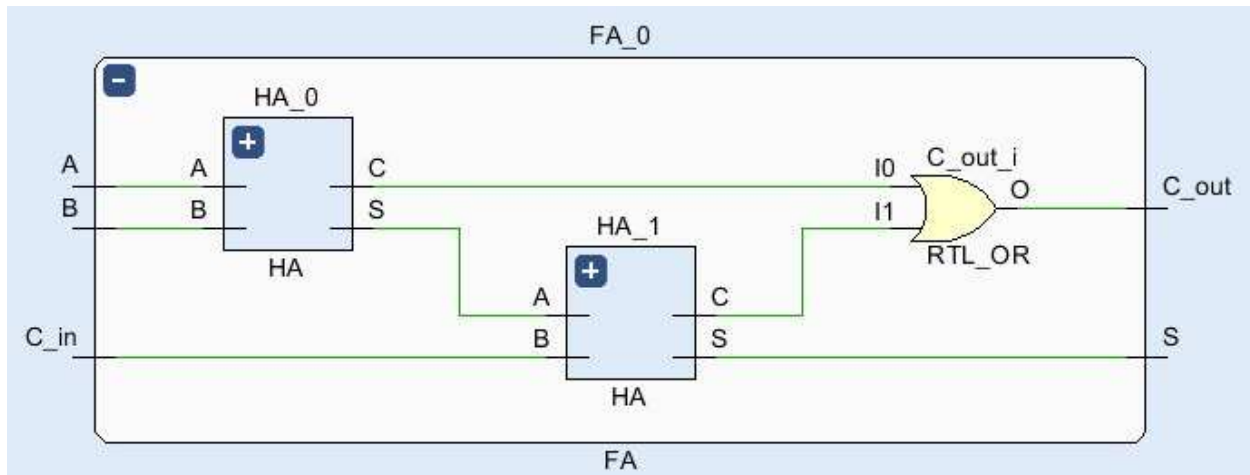


Full Adder

FA.vhd

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity FA is
5.     Port ( A : in STD_LOGIC;
6.           B : in STD_LOGIC;
7.           C_in : in STD_LOGIC;
8.           S : out STD_LOGIC;
9.           C_out : out STD_LOGIC);
10. end FA;
11.
12. architecture Behavioral of FA is
13.
14. component HA
15. port (
16. A: in std_logic;
17. B: in std_logic;
18. S: out std_logic;
19. C: out std_logic);
20. end component;
21.
22. SIGNAL HA0_S, HA0_C, HA1_S, HA1_C : std_logic;
23.
24. begin
25.
26. HA_0 : HA
27. port map (
28. A => A,
29. B => B,
30. S => HA0_S,
31. C => HA0_C);
32.
33. HA_1 : HA
34. port map (
35. A => HA0_S,
36. B => C_in,
37. S => HA1_S,
38. C => HA1_C);
39.
40. S <= HA1_S;
41. C_out <= HA0_C OR HA1_C;
42.
43.
44. end Behavioral;
45.
```

Design Schematic – FA



Half Adder

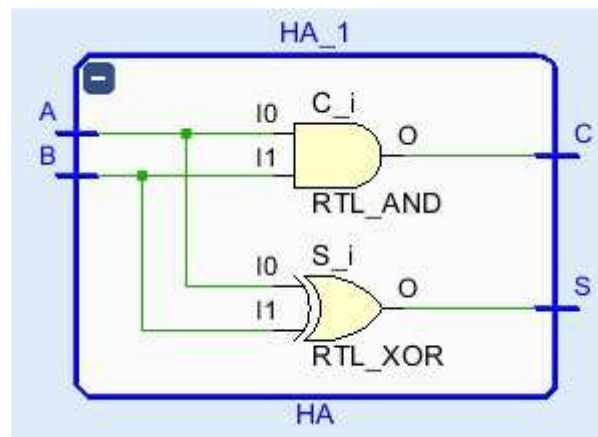
HA.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity HA is
5.     Port ( A : in STD_LOGIC;
6.           B : in STD_LOGIC;
7.           S : out STD_LOGIC;
8.           C : out STD_LOGIC);
9. end HA;
10.
11. architecture Behavioral of HA is
12.
13. begin
14.     S <= A XOR B;
15.     C <= A AND B;
16.
17.
18. end Behavioral;

```

Design Schematic – HA



Program ROM

Program_ROM.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity Program_ROM is
6.     port(ROM_address : in STD_LOGIC_VECTOR (2 downto 0);
7.          I: out STD_LOGIC_VECTOR (11 downto 0)
8.         );
9. end Program_ROM;
10.
11. architecture Behavioral of Program_ROM is
12.
13. type ROM_Type is array (0 to 7) of std_logic_vector(11 downto 0);
14.
15.     signal Program : ROM_Type := (
16.         "101110000011",    -- MOVI R7, 3
17.         "100010000001",    -- MOVI R1, 1

```

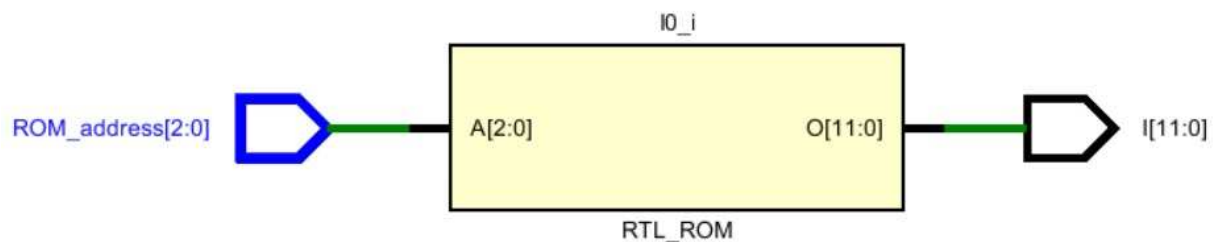


```

18.      "010010000000",      -- NEG  R1
19.      "100100000011",      -- MOVI R2, 3
20.      "000100010000",      -- ADD  R2, R1
21.      "001110100000",      -- ADD  R7, R2
22.      "110100000110",      -- JZR  R2, 6
23.      "110000000100"      -- JZR  R0, 4
24.
25.    );
26.
27.  begin
28.    process(ROM_address)
29.    begin
30.      if to_integer(unsigned(ROM_address)) < Program'length then
31.        I <= Program(to_integer(unsigned(ROM_address)));
32.      else
33.        I <= (others => '0');
34.      end if;
35.    end process;
36.  end Behavioral;
37.

```

Design Schematic – Program ROM



Display Driver

Display_Driver.vhd

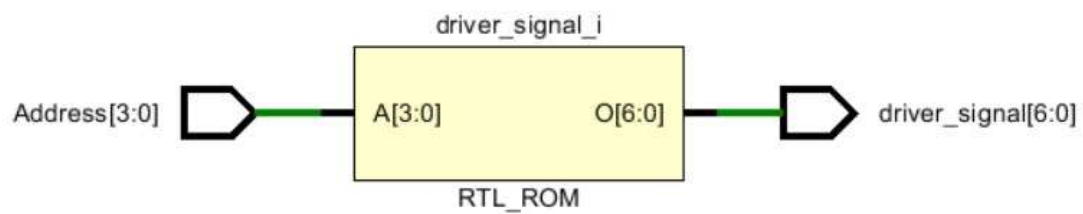
```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity Display_Driver is
6.     Port ( Address : in STD_LOGIC_VECTOR (3 downto 0);
7.           driver_signal : out STD_LOGIC_VECTOR (6 downto 0)
8.         );
9. end Display_Driver;
10.
11. architecture Behavioral of Display_Driver is
12.
13. Component MUX8 is
14.     Port (
15.         Sel    : in  STD_LOGIC_VECTOR(2 downto 0);
16.         I0     : in  STD_LOGIC_VECTOR(3 downto 0);
17.         I1     : in  STD_LOGIC_VECTOR(3 downto 0);
18.         I2     : in  STD_LOGIC_VECTOR(3 downto 0);
19.         I3     : in  STD_LOGIC_VECTOR(3 downto 0);
20.         I4     : in  STD_LOGIC_VECTOR(3 downto 0);
21.         I5     : in  STD_LOGIC_VECTOR(3 downto 0);
22.         I6     : in  STD_LOGIC_VECTOR(3 downto 0);
23.         I7     : in  STD_LOGIC_VECTOR(3 downto 0);
24.         O      : out STD_LOGIC_VECTOR(3 downto 0)
25.     );
26. end Component;
27.
28. type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
29.
30.     signal sevenSegment_ROM : rom_type := (
31.         "1000000", -- 0
32.         "1111001", -- 1
33.         "0100100", -- 2
34.         "0110000", -- 3
35.         "0011001", -- 4
36.         "0010010", -- 5
37.         "0000010", -- 6
38.         "1111000", -- 7
39.         "0000000", -- 8
40.         "0010000", -- 9
41.         "0001000", -- A
42.         "0000011", -- B
43.         "1000110", -- C
44.         "0100001", -- D
45.         "0000110", -- E
46.         "0001110"  -- F
47.     );
48. begin

```

```
49.  
50. driver_signal <= sevenSegment_ROM(to_integer(unsigned(Address)));  
51.  
52. end Behavioral;  
53.
```

Design Schematic – Driver



Simulation Files & Timing Diagrams

Computer

Test_Comp.vhd

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Test_Comp is
5. end Test_Comp;
6.
7. architecture Behavioral of Test_Comp is
8.
9.     component Computer is
10.     Port ( Clk : in STD_LOGIC;
11.           Reset : in STD_LOGIC;
12.           Seg7 : out STD_LOGIC_VECTOR (6 downto 0);
13.           data : out STD_LOGIC_VECTOR (3 downto 0);
14.           Zero : out STD_LOGIC;
15.           Overflow : out STD_LOGIC);
16.     end component;
17.
18.     -- Internal signals
19.     signal ClkIn : std_logic := '0';
20.     signal ResetIn : std_logic := '0';
21.     signal Overflow : std_logic;
22.     signal Zero : std_logic;
23.     signal seg : std_logic_vector(6 downto 0);
24.     signal led : std_logic_vector(3 downto 0);
25.
26. begin
27.
28.     -- Instantiate the Unit Under Test (UUT)
29.     uut: Computer
30.         port map (
31.             Clk => ClkIn,
32.             Reset => ResetIn,
33.             Seg7 => seg,
34.             data => led,
35.             Zero => Zero,
36.             Overflow => Overflow
37.         );
38.     ResetIn <= '0';
39.
40.     Clk_process : process
41.     begin
42.         while true loop
43.             ClkIn <= '0';
44.             wait for 5 ns;
45.             ClkIn <= '1';
```

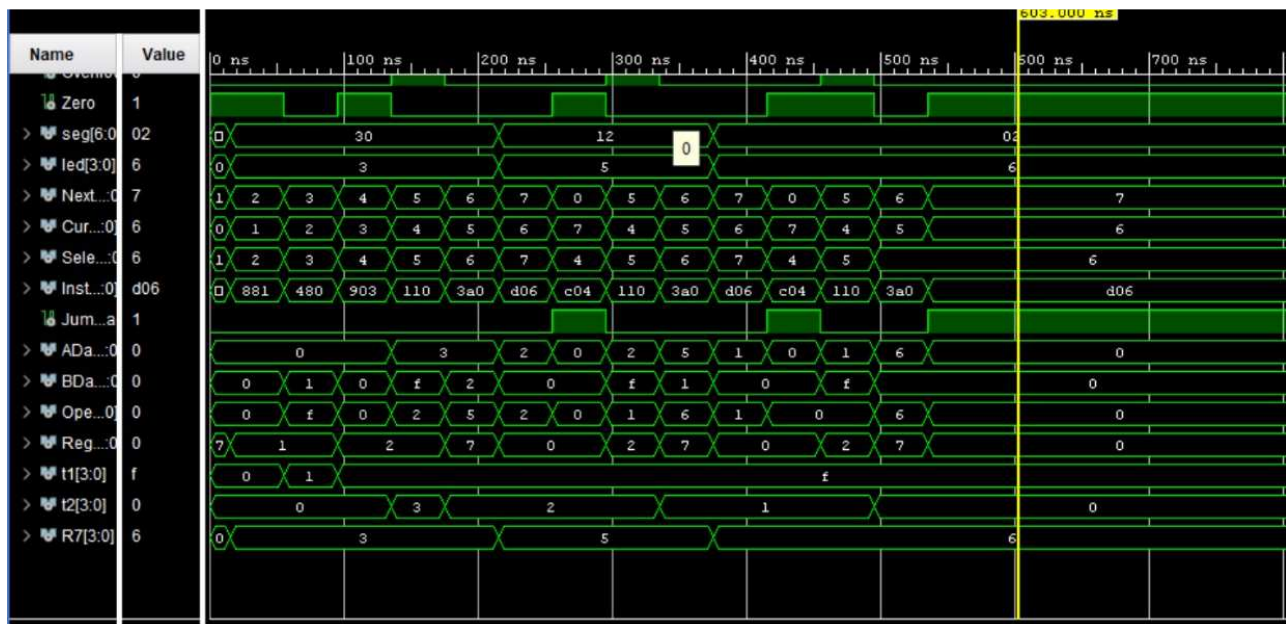
```

46.         wait for 5 ns;
47.     end loop;
48. end process;
49.
50. end Behavioral;
51.

```

Timing Diagram - Test_Comp

This includes the other components' intermediate signals as well



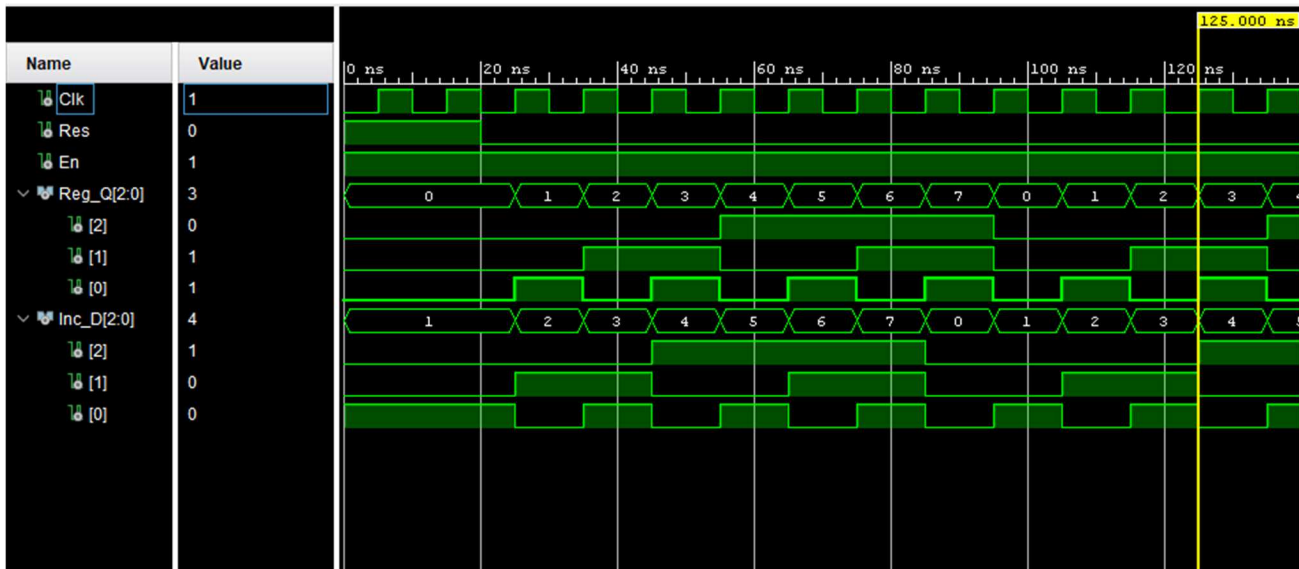
Program Counter

PC_Test.vhd

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity PC_Test is
5. end PC_Test;
6.
7. architecture Behavioral of PC_Test is
8.
9.     -- Component declaration for Register_PC
10.    component Register_PC is
11.        Port (
12.            Clk : in  STD_LOGIC;
13.            Res : in  STD_LOGIC;
14.            En  : in  STD_LOGIC;
15.            D   : in  STD_LOGIC_VECTOR(2 downto 0);
16.            Q   : out STD_LOGIC_VECTOR(2 downto 0)
17.        );
18.    end component;
19.
20.    -- Component declaration for PC_Inc
21.    component PC_Inc is
22.        Port (
23.            Q : in  STD_LOGIC_VECTOR(2 downto 0);
24.            D : out STD_LOGIC_VECTOR(2 downto 0)
25.        );
26.    end component;
27.
28.    -- Signals
29.    signal Clk      : STD_LOGIC := '0';
30.    signal Res      : STD_LOGIC := '0';
31.    signal En       : STD_LOGIC := '1';
32.    signal Reg_Q    : STD_LOGIC_VECTOR(2 downto 0);
33.    signal Inc_D    : STD_LOGIC_VECTOR(2 downto 0);
34.
35. begin
36.
37.     -- Instantiate Register_PC
38.     Reg : Register_PC
39.     port map (
40.         Clk => Clk,
41.         D   => Inc_D,
42.         En  => En,
43.         Res => Res,
44.         Q   => Reg_Q
45.     );
46.
47.     -- Instantiate PC_Inc
48.     PCI : PC_Inc
```

```
49.         port map (  
50.             Q => Reg_Q,  
51.             D => Inc_D  
52.         );  
53.  
54.         -- Clock process (10 ns period)  
55.         Clock : process  
56.         begin  
57.             while true loop  
58.                 Clk <= '0';  
59.                 wait for 5 ns;  
60.                 Clk <= '1';  
61.                 wait for 5 ns;  
62.             end loop;  
63.         end process Clock;  
64.  
65.         -- Stimulus process  
66.         Stimulus : process  
67.         begin  
68.             -- Reset initially  
69.             Res <= '1';  
70.             wait for 20 ns;  
71.             Res <= '0';  
72.  
73.  
74.             wait for 100 ns;  
75.  
76.             -- Stop simulation  
77.             wait;  
78.         end process Stimulus;  
79.  
80.     end Behavioral;  
81.
```

Timing Diagram - PC_Test



3 to 8 Decoder

TB_Decoder_3_to_8.vhd

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity TB_Decoder_3_to_8 is
5. end TB_Decoder_3_to_8;
6.
7. architecture Behavioral of TB_Decoder_3_to_8 is
8. COMPONENT Decoder_8
9. port( Sel: in STD_LOGIC_VECTOR;
10.       O: out STD_LOGIC_VECTOR );
11. END COMPONENT;
12.
13. SIGNAL I: STD_LOGIC_VECTOR (2 downto 0);
14. SIGNAL Y: STD_LOGIC_VECTOR (7 downto 0);
15.
16. begin
17.
18. UUT: Decoder_8
19. PORT MAP(
20.     Sel => I,
21.     O => Y );
22.
23. process
24. begin

```

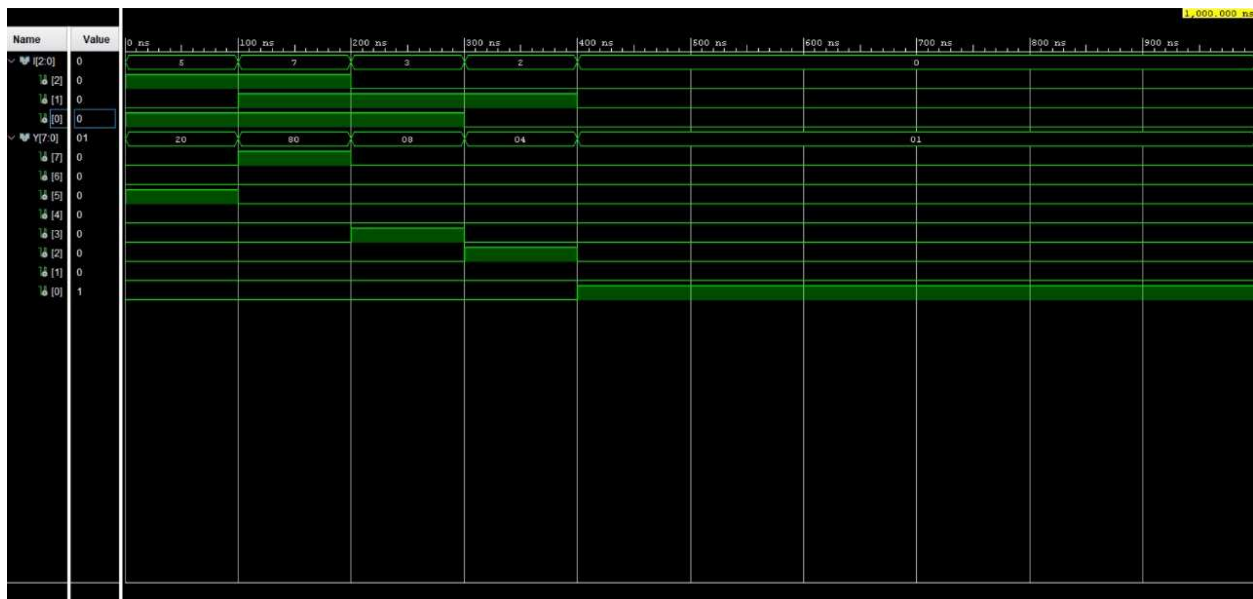


```

25.
26. I(2) <= '1'; I(1) <= '0'; I(0) <= '1';
27. WAIT FOR 100 ns;
28.
29. I(2) <= '1'; I(1) <= '1'; I(0) <= '1';
30. WAIT FOR 100 ns;
31.
32. I(2) <= '0'; I(1) <= '1'; I(0) <= '1';
33. WAIT FOR 100 ns;
34.
35. I(2) <= '0'; I(1) <= '1'; I(0) <= '0';
36. WAIT FOR 100 ns;
37.
38. I(2) <= '0'; I(1) <= '0'; I(0) <= '0';
39. WAIT;
40.
41. end process;
42.
43. end Behavioral;
44.

```

Timing Diagram - TB_Decoder_3_to_8



Arithmetic Unit

Test_AU.vhd

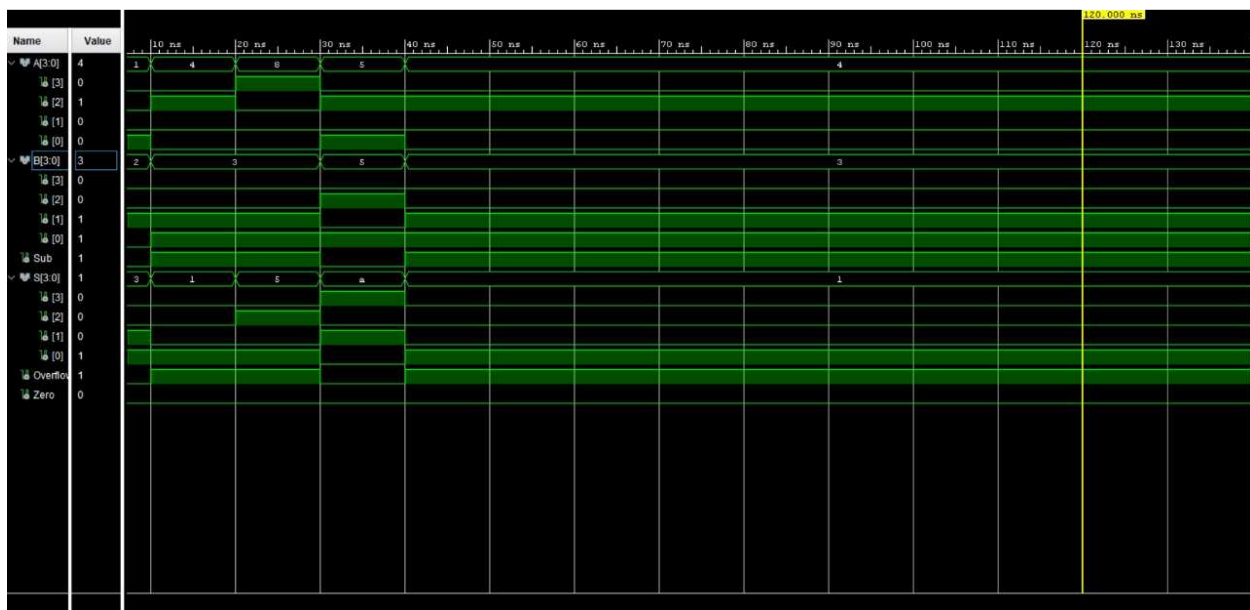
```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Test_AU is
5. end Test_AU ;
6.
7. architecture Behavioral of Test_AU is
8.
9.     signal A, B : STD_LOGIC_VECTOR(3 downto 0);
10.    signal Sub : STD_LOGIC;
11.    signal S : STD_LOGIC_VECTOR(3 downto 0);
12.    signal Overflow : STD_LOGIC;
13.    signal Zero : STD_LOGIC;
14.
15.    component AU is
16.        Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
17.              B : in STD_LOGIC_VECTOR (3 downto 0);
18.              Sub : in STD_LOGIC;
19.              S : out STD_LOGIC_VECTOR (3 downto 0);
20.              Overflow : out STD_LOGIC;
21.              Zero : out STD_LOGIC);
22.    end component;
23.
24. begin
25.
26.    UUT: AU
27.        port map (
28.            A => A,
29.            B => B,
30.            Sub => Sub,
31.            S => S,
32.            Overflow => Overflow,
33.            Zero => Zero
34.        );
35.
36.
37.    process
38.    begin
39.
40.        A <= "0001";
41.        B <= "0010";
42.        Sub <= '0';
43.        wait for 10 ns;
44.
45.
46.        A <= "0100";
47.        B <= "0011";
48.        Sub <= '1';
```

```

49.      wait for 10 ns;
50.
51.
52.      A <= "0011";
53.      B <= "0011";
54.      Sub <= '1';
55.      wait for 10 ns;
56.
57.      wait;
58.  end process;
59. end Behavioral;
60.

```

Timing Diagram - TB_AU



Instruction Decoder

Test_ID (Test_ID.vhd)

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity Test_ID is
5. end Test_ID;
6.
7. architecture behavior of Test_ID is
8.
9.     -- Component Declaration for the Unit Under Test (UUT)
10.    component Instruction_Decoder
11.        Port (
12.            Instruction    : in  STD_LOGIC_VECTOR(11 downto 0);
13.            JumpCheck     : in  STD_LOGIC_VECTOR(3 downto 0);
14.            RegEn         : out STD_LOGIC_VECTOR(2 downto 0);
15.            Sel_A         : out STD_LOGIC_VECTOR(2 downto 0);
16.            Sel_B         : out STD_LOGIC_VECTOR(2 downto 0);
17.            LoadSel       : out STD_LOGIC;
18.            ImmVal        : out STD_LOGIC_VECTOR(3 downto 0);
19.            Sub           : out STD_LOGIC;
20.            JumpFlag      : out STD_LOGIC;
21.            JumpAddress   : out STD_LOGIC_VECTOR(2 downto 0)
22.        );
23.    end component;
24.
25.    -- Signals to connect to the UUT
26.    signal Instruction    : STD_LOGIC_VECTOR(11 downto 0);
27.    signal JumpCheck     : STD_LOGIC_VECTOR(3 downto 0);
28.    signal RegEn         : STD_LOGIC_VECTOR(2 downto 0);
29.    signal Sel_A         : STD_LOGIC_VECTOR(2 downto 0);
30.    signal Sel_B         : STD_LOGIC_VECTOR(2 downto 0);
31.    signal LoadSel       : STD_LOGIC;
32.    signal ImmVal        : STD_LOGIC_VECTOR(3 downto 0);
33.    signal Sub           : STD_LOGIC;
34.    signal JumpFlag      : STD_LOGIC;
35.    signal JumpAddress   : STD_LOGIC_VECTOR(2 downto 0);
36.
37. begin
38.
39.     -- Instantiate the Unit Under Test (UUT)
40.    uut: Instruction_Decoder
41.        Port map (
42.            Instruction    => Instruction,
43.            JumpCheck     => JumpCheck,
44.            RegEn         => RegEn,
45.            Sel_A         => Sel_A,
46.            Sel_B         => Sel_B,
47.            LoadSel       => LoadSel,
48.            ImmVal        => ImmVal,

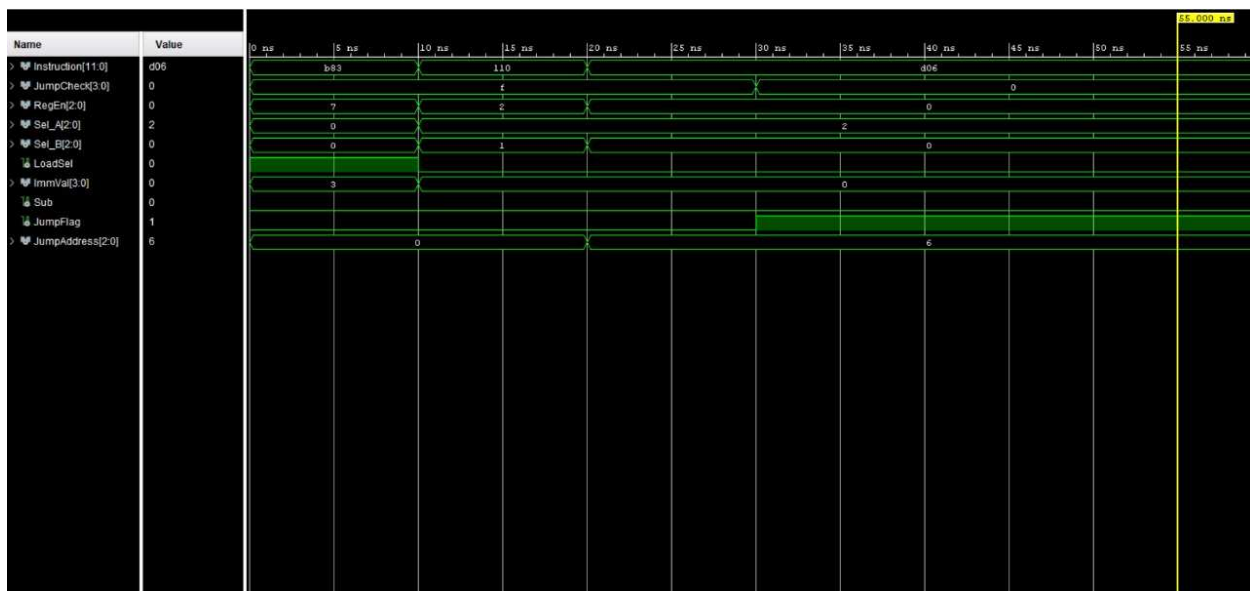
```

```

49.         Sub           => Sub,
50.         JumpFlag      => JumpFlag,
51.         JumpAddress    => JumpAddress
52.     );
53.
54.     -- Stimulus process
55.     stim_proc: process
56.     begin
57.         JumpCheck <= "1111";
58.
59.         -- MOVI R7, 3
60.         Instruction <= "101110000011";
61.         Wait for 10 ns;
62.
63.         -- ADD R2, R1
64.         Instruction <= "000100010000";
65.         Wait for 10 ns;
66.
67.         -- JZR R2, 6
68.         Instruction <= "110100000110";
69.         Wait for 10 ns;
70.
71.         -- JZR R2, 6
72.         JumpCheck <= "0000";
73.         Instruction <= "110100000110";
74.         Wait for 10 ns;
75.
76.         wait;
77.     end process;
78. end behavior;
79.

```

Timing Diagram - TB_ID



Register Bank

Test_RB (Test_RB.vhd)

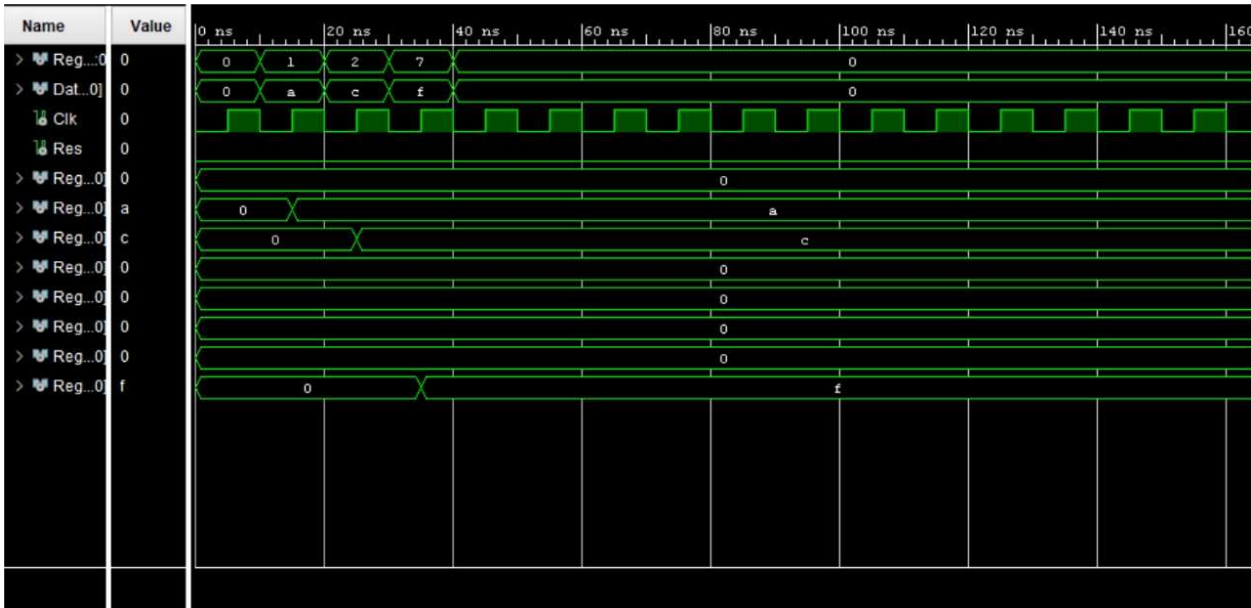
```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.STD_LOGIC_UNSIGNED.ALL;
4.
5. entity Test_RB is
6. end Test_RB;
7.
8. architecture Behavioral of Test_RB is
9.
10.
11.     component Register_Bank
12.         Port (
13.             RegEn    : in  STD_LOGIC_VECTOR(2 downto 0);
14.             Data     : in  STD_LOGIC_VECTOR(3 downto 0);
15.             Clk      : in  STD_LOGIC;
16.             Reg0     : out STD_LOGIC_VECTOR(3 downto 0);
17.             Reg1     : out STD_LOGIC_VECTOR(3 downto 0);
18.             Reg2     : out STD_LOGIC_VECTOR(3 downto 0);
19.             Reg3     : out STD_LOGIC_VECTOR(3 downto 0);
20.             Reg4     : out STD_LOGIC_VECTOR(3 downto 0);
21.             Reg5     : out STD_LOGIC_VECTOR(3 downto 0);
22.             Reg6     : out STD_LOGIC_VECTOR(3 downto 0);
23.             Reg7     : out STD_LOGIC_VECTOR(3 downto 0);
24.         );
25.     end component;
26.
27.     -- Test signals
28.     signal RegEn    : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');
29.     signal Data     : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
30.     signal Clk      : STD_LOGIC := '0';
31.     signal Reg0, Reg1, Reg2, Reg3, Reg4, Reg5, Reg6, Reg7 : STD_LOGIC_VECTOR(3 downto 0);
32.
33. begin
34.
35.     -- Instantiate the Unit Under Test (UUT)
36.     UUT: Register_Bank
37.         port map (
38.             RegEn => RegEn,
39.             Data  => Data,
40.             Clk   => Clk,
41.             Reg0  => Reg0,
42.             Reg1  => Reg1,
43.             Reg2  => Reg2,
44.             Reg3  => Reg3,
45.             Reg4  => Reg4,
46.             Reg5  => Reg5,
47.             Reg6  => Reg6,
48.             Reg7  => Reg7
49.         );
50.
51.     -- Clock process
52.     clk_process : process
53.     begin
54.         Clk <= '0';

```

```
55.         wait for 5 ns;
56.         Clk <= '1';
57.         wait for 5 ns;
58.     end process;
59.
60.     -- Stimulus process
61.     stim_proc: process
62.     begin
63.         -- Wait a bit before starting
64.         wait for 10 ns;
65.
66.         -- Write 1010 to Reg1
67.         Data <= "1010";
68.         RegEn <= "001"; -- Select Reg1
69.         wait for 10 ns;
70.
71.         -- Write 1100 to Reg2
72.         Data <= "1100";
73.         RegEn <= "010"; -- Select Reg2
74.         wait for 10 ns;
75.
76.         -- Write 1111 to Reg7
77.         Data <= "1111";
78.         RegEn <= "111"; -- Select Reg7
79.         wait for 10 ns;
80.
81.         -- Stop writing
82.         RegEn <= "000";
83.         Data <= "0000";
84.         wait for 10 ns;
85.
86.         -- Done
87.         wait;
88.     end process;
89.
90. end Behavioral;
91.
```

Timing Diagram - Test_RB



Constraints File (Basys3.xdc)

```
1.  ## Clock signal
2.  set_property PACKAGE_PIN W5 [get_ports Clk]
3.  set_property IOSTANDARD LVCMOS33 [get_ports Clk]
4.  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports Clk]
5.
6.
7.  # LEDs
8.  set_property PACKAGE_PIN U16 [get_ports {data[0]}]
9.  set_property IOSTANDARD LVCMOS33 [get_ports {data[0]}]
10. set_property PACKAGE_PIN E19 [get_ports {data[1]}]
11. set_property IOSTANDARD LVCMOS33 [get_ports {data[1]}]
12. set_property PACKAGE_PIN U19 [get_ports {data[2]}]
13. set_property IOSTANDARD LVCMOS33 [get_ports {data[2]}]
14. set_property PACKAGE_PIN V19 [get_ports {data[3]}]
15. set_property IOSTANDARD LVCMOS33 [get_ports {data[3]}]
16. set_property PACKAGE_PIN P1 [get_ports {Overflow}]
17. set_property IOSTANDARD LVCMOS33 [get_ports {Overflow}]
18. set_property PACKAGE_PIN L1 [get_ports {Zero}]
19. set_property IOSTANDARD LVCMOS33 [get_ports {Zero}]
20.
21.
22. #7 segment display
23. set_property PACKAGE_PIN W7 [get_ports {Seg7[0]}]
24. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[0]}]
25. set_property PACKAGE_PIN W6 [get_ports {Seg7[1]}]
26. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[1]}]
27. set_property PACKAGE_PIN U8 [get_ports {Seg7[2]}]
28. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[2]}]
29. set_property PACKAGE_PIN V8 [get_ports {Seg7[3]}]
30. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[3]}]
31. set_property PACKAGE_PIN U5 [get_ports {Seg7[4]}]
32. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[4]}]
33. set_property PACKAGE_PIN V5 [get_ports {Seg7[5]}]
34. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[5]}]
35. set_property PACKAGE_PIN U7 [get_ports {Seg7[6]}]
36. set_property IOSTANDARD LVCMOS33 [get_ports {Seg7[6]}]
37.
38.
39.
40. set_property PACKAGE_PIN U2 [get_ports {Anode[0]}]
41. set_property IOSTANDARD LVCMOS33 [get_ports {Anode[0]}]
42. set_property PACKAGE_PIN U4 [get_ports {Anode[1]}]
43. set_property IOSTANDARD LVCMOS33 [get_ports {Anode[1]}]
44. set_property PACKAGE_PIN V4 [get_ports {Anode[2]}]
45. set_property IOSTANDARD LVCMOS33 [get_ports {Anode[2]}]
46. set_property PACKAGE_PIN W4 [get_ports {Anode[3]}]
47. set_property IOSTANDARD LVCMOS33 [get_ports {Anode[3]}]
48. ##Buttons
49. set_property PACKAGE_PIN U18 [get_ports Reset]
50. set_property IOSTANDARD LVCMOS33 [get_ports Reset]
51.
```

Section 2

Optimized Nano Processor

Objective

All optimizations were carried out with the primary objective of reducing the LUT (Look-Up Table) usage in the implemented design. While this approach may not perfectly reflect the minimal use of logic gates in the Nano-Processor architecture or represent the most efficient implementation on an FPGA, it aligns with the goal of minimizing resource utilization within the given design constraints.

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	DSP s (90)	Bonded IOB (106)
Computer	15	12	10	15	3	19
> NP1 (NanoProcessor)	15	12	10	15	2	0
> SlowClk (Slow_Clk_Bin)	0	0	0	0	1	0

Fig 2.2.1: Utilization Report for the Optimized Nano Processor

Methodology

Further optimization from the base model was done by utilizing DSP Slices and manually assigning CARRY4 primitives. Utilization of DSP blocks were mainly done through the IP catalog. The notable changes done are as follows,

- The existing Adder/Subtractor unit which was using a ripple carry adder was replaced by unsigned 5-bit Adder/Subtractor unit with a carry out from the IP catalog which utilizes DSP48 block.
- PC Register with its constituents was replaced by a loadable Binary Counter with a synchronous reset from the IP catalog which also utilizes DSP48 block.
- Slow Clock was also replaced by a 27- bit Binary Counter and it's MSB was used as the slowed clock signal.
- Zero-flag logic was custom-built using CARRY4 primitives. 2 NOT gates were efficiently implemented using a single CARRY4 block. Additionally, logic to assert a flag when all inputs are high was realized using another CARRY4 block. These components were then combined to generate the final zero-flag signal.

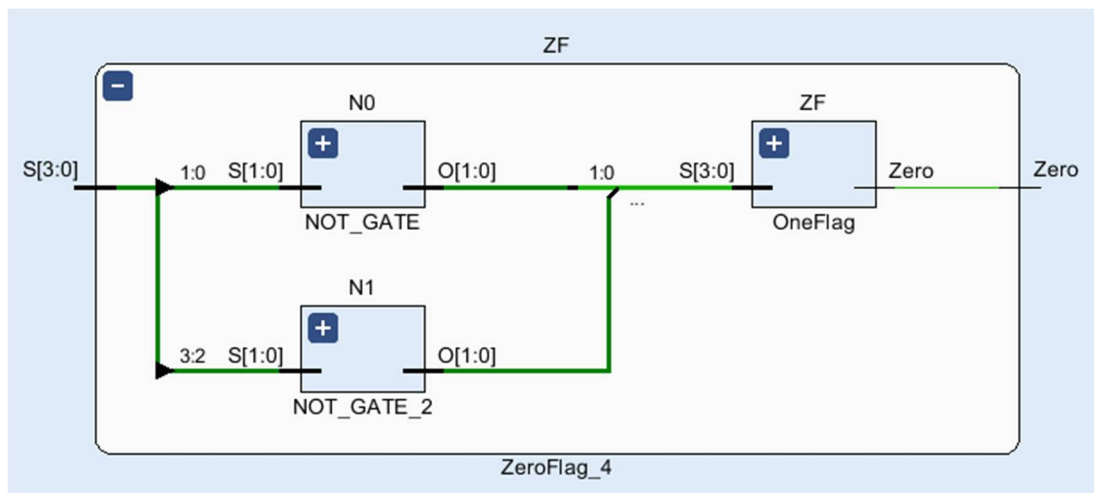


Fig 2.3.1: Schematic of the Zero-Flag

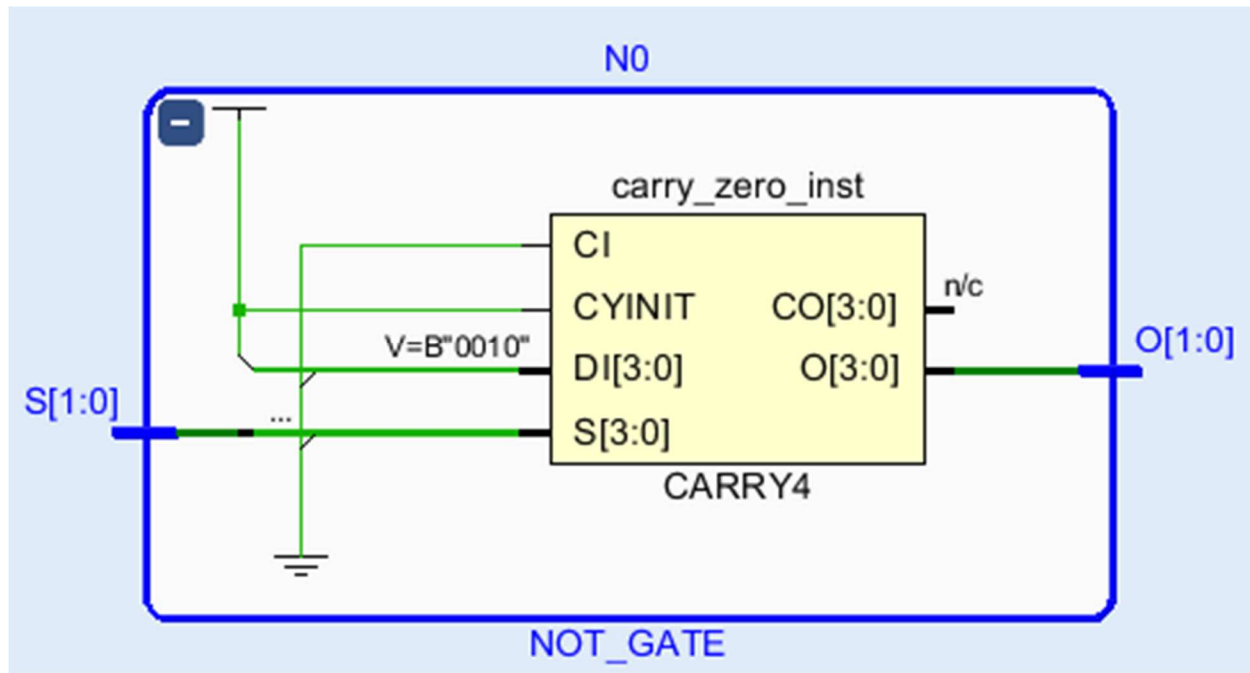


Fig 2.3.1: Schematic of the Not Gate

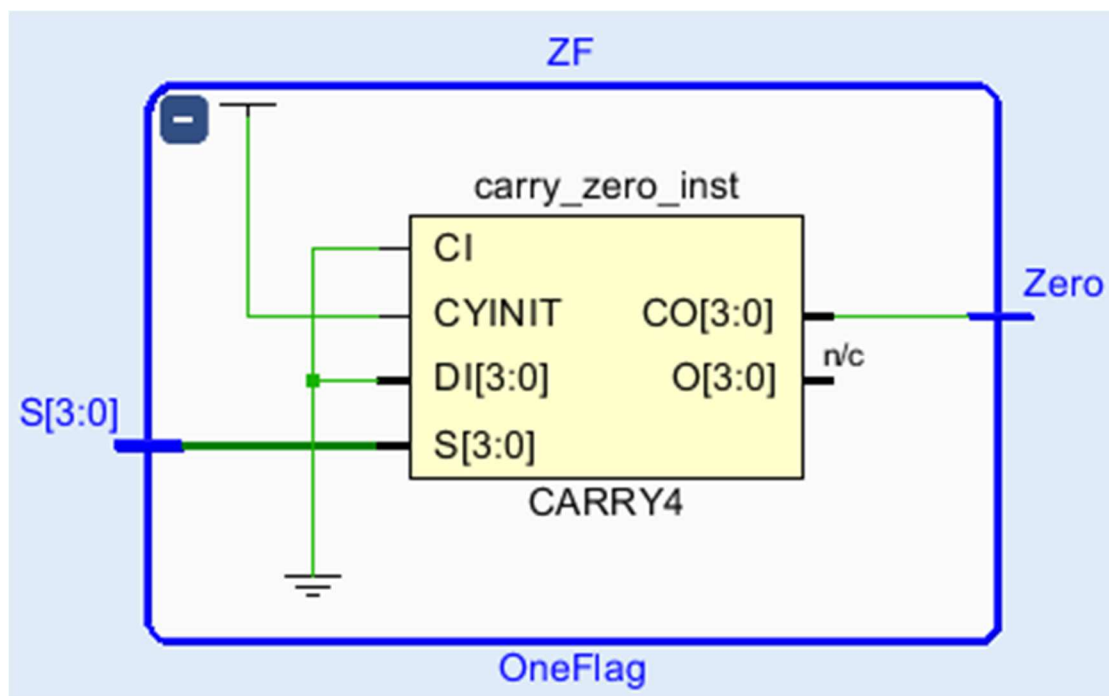


Fig 2.3.3: Schematic of the One Flag

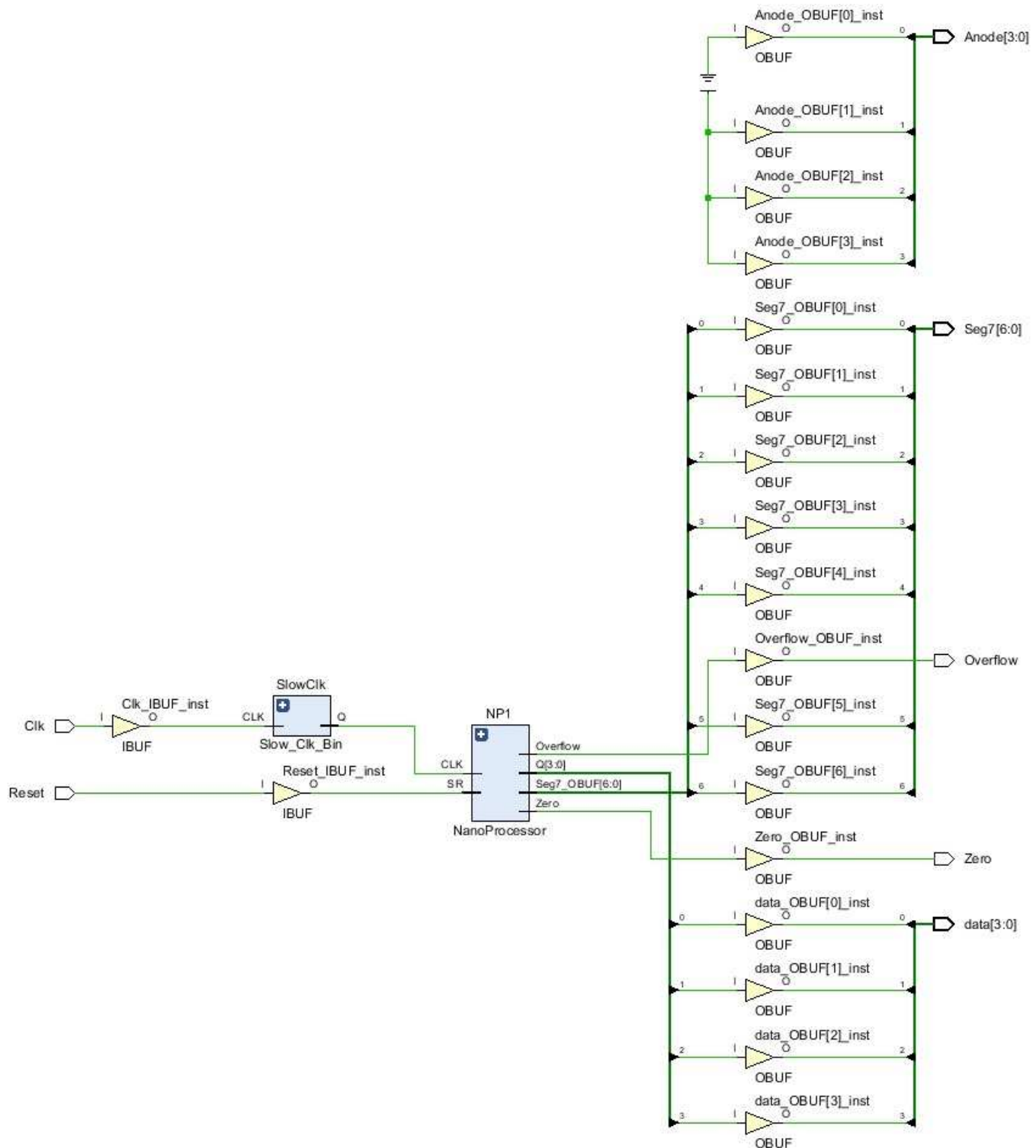


Fig 2.3.4: Implemented Design Schematic of the Nano processor Design

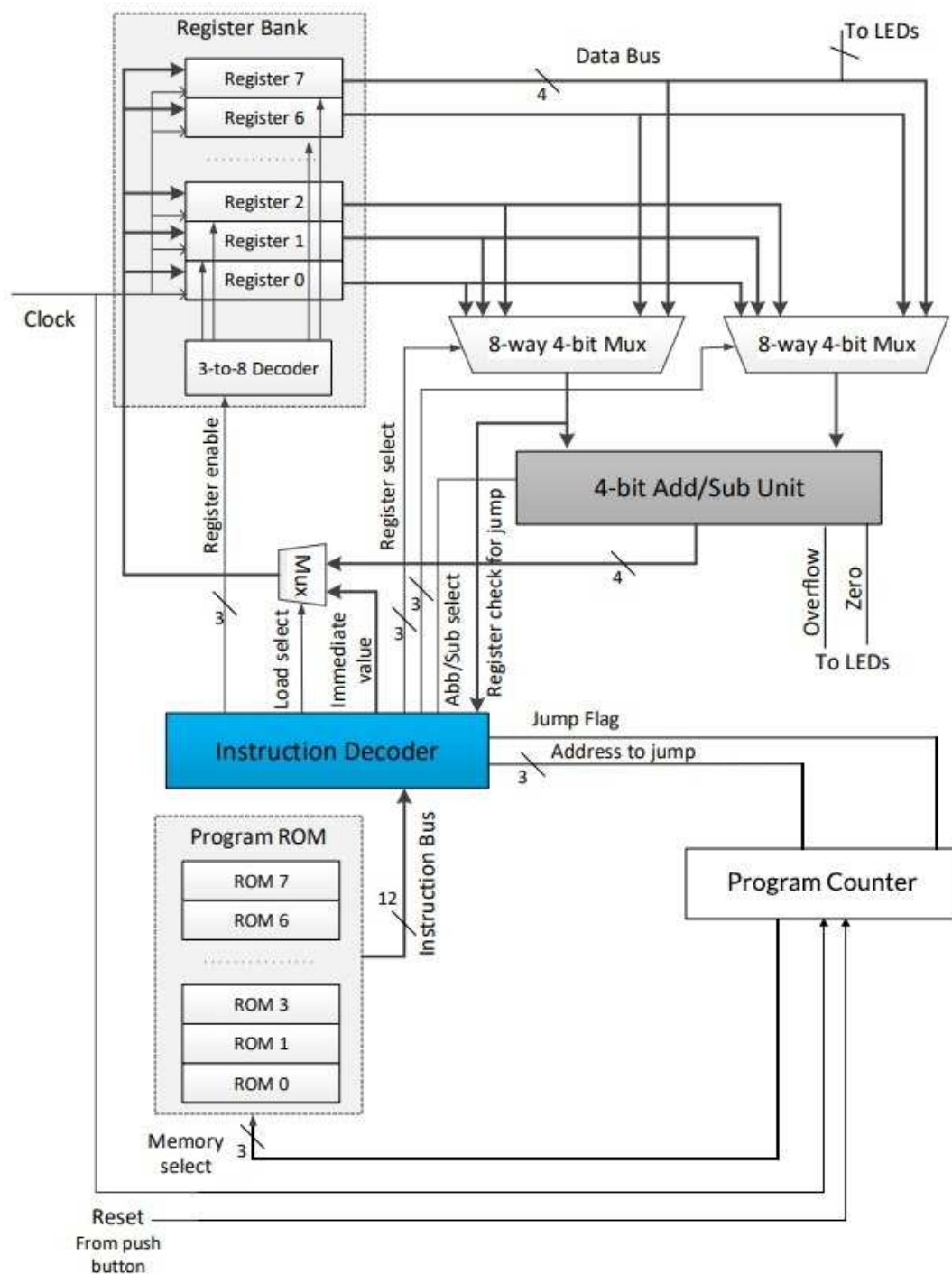


Fig 2.3.5: Abstraction of the Optimized Nano processor

Section 3

Extended Nano Processor

Objective

The objective of the Extended Nano Processor project is to expand upon the functionality of the base nano processor by adding new features and capabilities that support a wider range of operations and control logic. The focus is on achieving correct and complete functionality through the integration of additional components and modules, without prioritizing resource efficiency or performance optimization. The enhancements aim to demonstrate the extensibility of the processor design and enable the execution of more complex instructions and workflows.

Methodology

The processor has been upgraded to an 8-bit architecture, enabling it to handle 8-bit data operations and expand its computational capabilities. In addition to the architectural enhancement, support for a total of eight distinct instructions has been implemented, allowing for a broader and more versatile instruction set. Furthermore, display-related features have been integrated to provide visual output capabilities, enhancing the processor's usability and enabling basic user interaction through visual feedback.

The following are some of the notable additional features,

- Changed negation (NEG) instruction to subtraction (SUB) which extends the capabilities of the negation instruction.
- Increased the number of instructions which includes multiplication, division, comparison and bitwise operations.
- Added 4 different program ROMs that the user can select.
- Added a step mode which the user can switch to instead of the normal clock mode.
- Added a register select where the user can pick what register's value that needs to be displayed.
- Added Multiple display modes that user can select and handle signed values if needed (via user input)
 1. Program Counter and register value with 2 hex values.
 2. Base 10 value display using 3-bits.

	Opcode			Operand												
MOV	0	1	0	R _a	R _a	R _a	x	x	x	d	d	d	d	d	d	d
ADD	0	0	0	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x	x	x	x
SUB	0	0	1	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x	x	x	x
JZR	0	1	1	R _a	R _a	R _a	x	x	x	x	x	x	x	J	J	J
MUL	1	0	0	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x	x	x	x
DIV	1	0	1	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x	x	x	Q
COM	1	1	0	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x	x	x	L
BIT	1	1	1	R _a	R _a	R _a	R _b	R _b	R _b	x	x	x	x	x	b	b

Fig 3.2.1: Extended Instruction Set

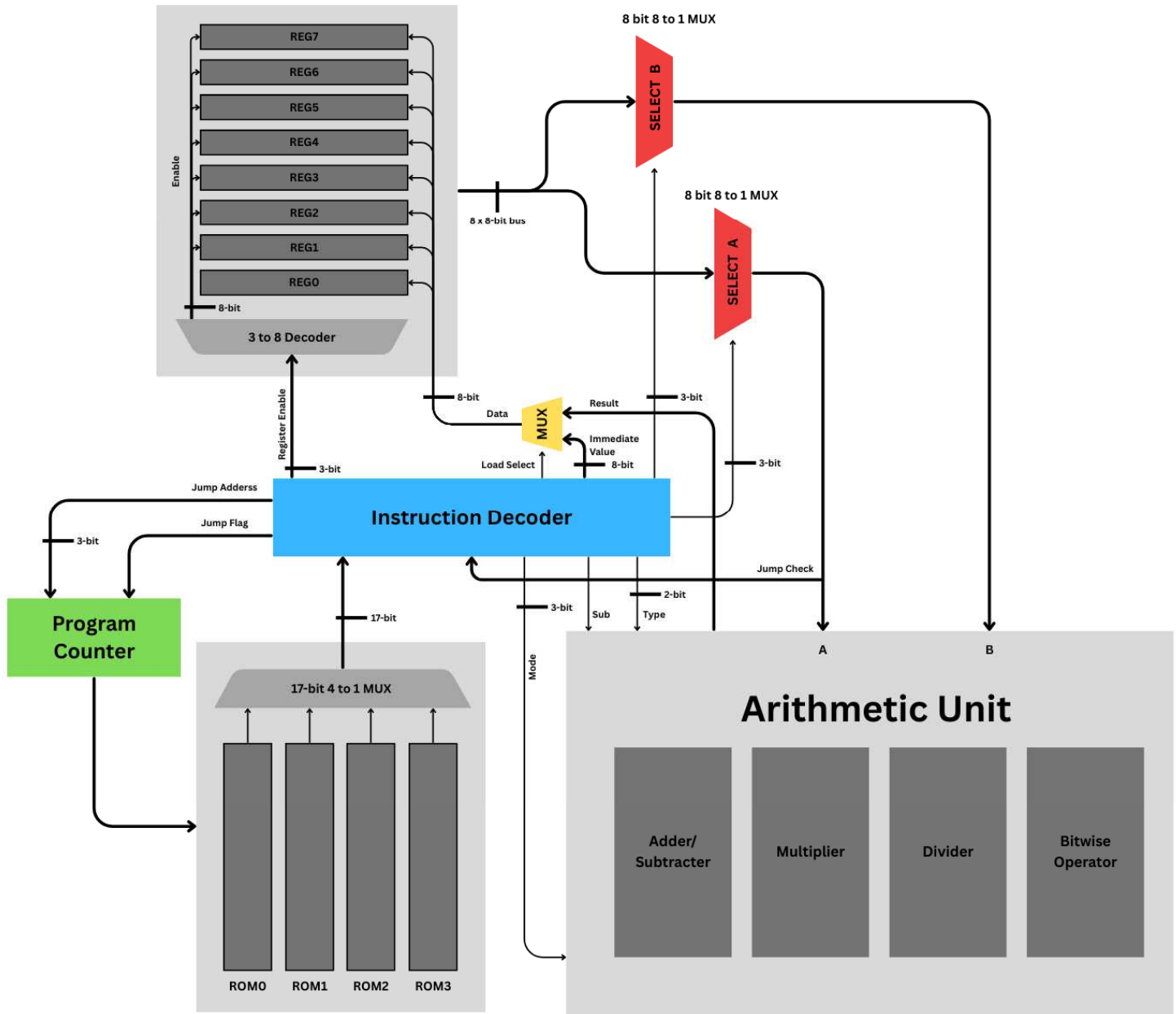


Fig 3.2.2: Abstraction of the Extended Nano Processor

Extended Instruction Decoder

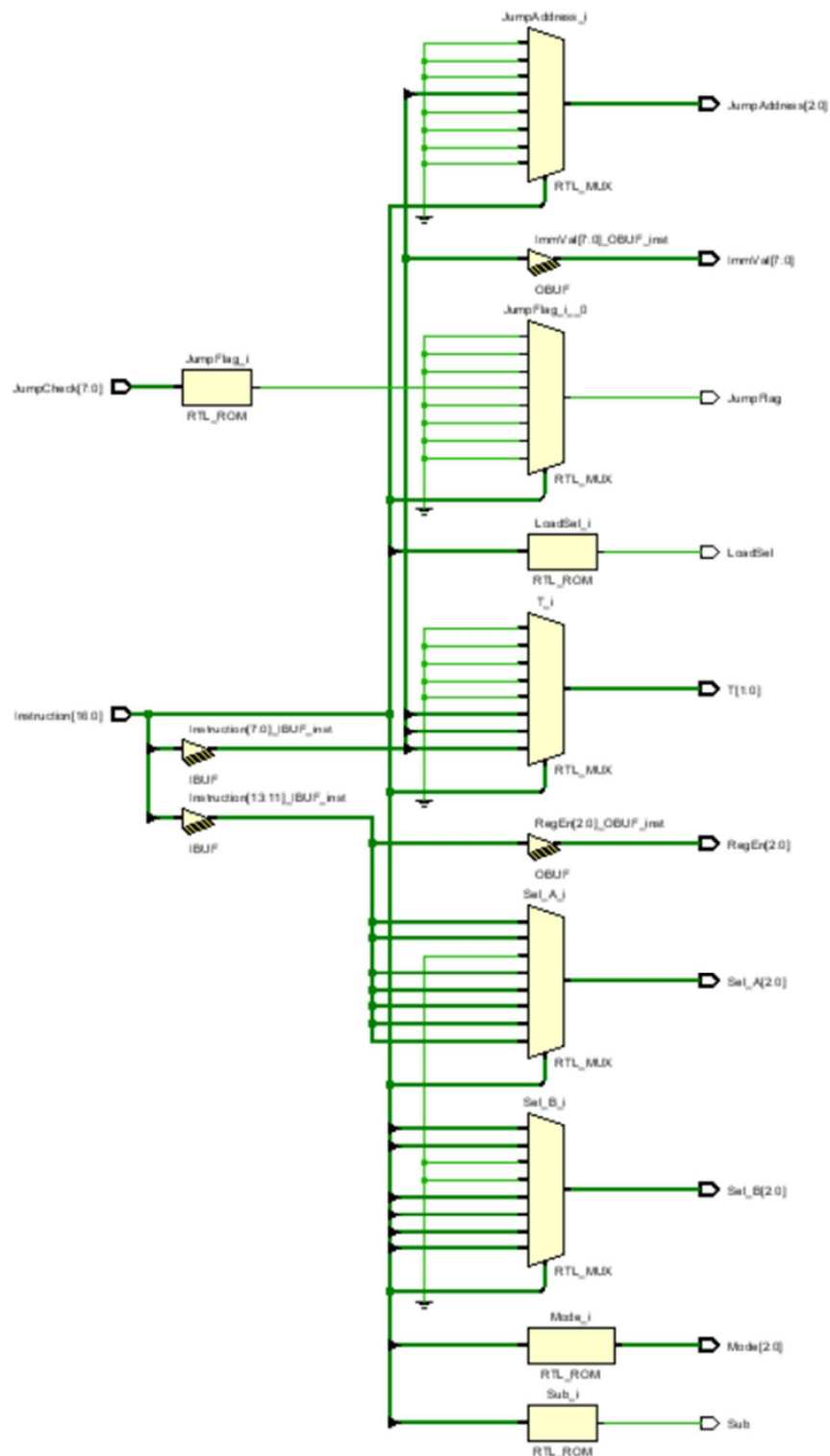
```

1.
2. library IEEE;
3. use IEEE.STD_LOGIC_1164.ALL;
4.
5. entity Instruction_Decoder is
6.     Port (
7.         Instruction : in  STD_LOGIC_VECTOR(16 downto 0);
8.         JumpCheck   : in  STD_LOGIC_VECTOR(7 downto 0);
9.         RegEn        : out STD_LOGIC_VECTOR(2 downto 0);
10.        Sel_A        : out STD_LOGIC_VECTOR(2 downto 0);
11.        Sel_B        : out STD_LOGIC_VECTOR(2 downto 0);
12.        LoadSel      : out STD_LOGIC;
13.        ImmVal       : out STD_LOGIC_VECTOR(7 downto 0);
14.        Sub          : out STD_LOGIC;
15.        JumpFlag     : out STD_LOGIC;
16.        JumpAddress  : out STD_LOGIC_VECTOR(2 downto 0);
17.
18.        Mode : out STD_LOGIC_VECTOR (2 downto 0);
19.        T : out STD_LOGIC_VECTOR (1 downto 0)
20.    );
21. end Instruction_Decoder;
22.
23. architecture Behavioral of Instruction_Decoder is
24.     signal opcode : STD_LOGIC_VECTOR(2 downto 0);
25. begin
26.
27.     opcode <= Instruction(16 downto 14);
28.
29.     process(opcode, Instruction, JumpCheck)
30.     begin
31.         -- Default outputs
32.         RegEn <= (others => '0');
33.         Sel_A <= (others => '0');
34.         Sel_B <= (others => '0');
35.         LoadSel <= '0';
36.         ImmVal <= Instruction(7 downto 0);
37.         Sub <= '0';
38.         JumpFlag <= '0';
39.         T <= "00";
40.         Mode <= (others => '0');
41.         JumpAddress <= (others => '0');
42.
43.         case opcode is
44.             when "000" => -- ADD
45.                 RegEn <= Instruction(13 downto 11);
46.                 Sel_A <= Instruction(13 downto 11);
47.                 Sel_B <= Instruction(10 downto 8);
48.                 Mode <= "000";
49.
50.             when "001" => -- SUB
51.                 RegEn <= Instruction(13 downto 11);
52.                 Sel_A <= Instruction(13 downto 11);
53.                 Sel_B <= Instruction(10 downto 8);
54.                 Mode <= "000";
55.                 Sub <= '1';
56.
57.             when "010" => -- MOVI (

```

```
58.         RegEn  <= Instruction(13 downto 11);
59.         LoadSel <= '1';
60.         ImmVal  <= Instruction(7 downto 0);
61.
62.     when "011" => -- JMZ
63.         RegEn <= Instruction(13 downto 11);
64.         Sel_A  <= Instruction(13 downto 11);
65.         JumpAddress <= Instruction(2 downto 0);
66.         if JumpCheck = "00000000" then
67.             JumpFlag <= '1';
68.         end if;
69.
70.     when "100" => -- Mult
71.         RegEn <= Instruction(13 downto 11);
72.         Sel_A <= Instruction(13 downto 11);
73.         Sel_B <= Instruction(10 downto 8);
74.         Mode  <= "001";
75.
76.     when "101" => -- Div
77.         RegEn <= Instruction(13 downto 11);
78.         Sel_A <= Instruction(13 downto 11);
79.         Sel_B <= Instruction(10 downto 8);
80.         Mode  <= "010";
81.         T <= Instruction(1 downto 0);
82.
83.     when "110" => -- Com (Divide)
84.         RegEn <= Instruction(13 downto 11);
85.         Sel_A <= Instruction(13 downto 11);
86.         Sel_B <= Instruction(10 downto 8);
87.         Mode  <= "011";
88.         T <= Instruction(1 downto 0);
89.
90.     when "111" => -- BITWISE
91.         RegEn <= Instruction(13 downto 11);
92.         Sel_A <= Instruction(13 downto 11);
93.         Sel_B <= Instruction(10 downto 8);
94.         Mode  <= "100";
95.         T <= Instruction(1 downto 0);
96.     when others =>
97.         null;
98.     end case;
99. end process;
100. end Behavioral;
101.
```

Design Schematic – Extended Instruction Decoder



Extended Arithmetic Unit

```

1.
2. library IEEE;
3. use IEEE.STD_LOGIC_1164.ALL;
4.
5. entity AU is
6.     Port ( Mode : in STD_LOGIC_VECTOR (2 downto 0);
7.           T : in STD_LOGIC_VECTOR (1 downto 0);
8.
9.           A : in STD_LOGIC_VECTOR (7 downto 0);
10.          B : in STD_LOGIC_VECTOR (7 downto 0);
11.          Sub : in STD_LOGIC;
12.
13.          S : out STD_LOGIC_VECTOR (7 downto 0);
14.          Overflow : out STD_LOGIC;
15.          Zero : out STD_LOGIC);
16. end AU;
17.
18. architecture Behavioral of AU is
19.
20. COMPONENT c_addsub_0 IS
21.     PORT (
22.         A : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
23.         B : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
24.         ADD : IN STD_LOGIC;
25.         C_OUT : OUT STD_LOGIC;
26.         S : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
27.     );
28. END COMPONENT;
29.
30. Component Divider is
31.     Port (
32.         A : in STD_LOGIC_VECTOR(7 downto 0);
33.         B : in STD_LOGIC_VECTOR(7 downto 0);
34.         S : out STD_LOGIC_VECTOR(7 downto 0);
35.         C : out STD_LOGIC_VECTOR(7 downto 0)
36.     );
37. end Component;
38.
39. Component Multiplier is
40.     Port ( A : in STD_LOGIC_VECTOR (7 downto 0);
41.           B : in STD_LOGIC_VECTOR (7 downto 0);
42.           S : out STD_LOGIC_VECTOR (7 downto 0));
43. end Component;
44.
45. Component Comparator is
46.     Port (
47.         A : in STD_LOGIC_VECTOR (7 downto 0);
48.         B : in STD_LOGIC_VECTOR (7 downto 0);
49.         G : out STD_LOGIC_VECTOR (7 downto 0);
50.         L : out STD_LOGIC_VECTOR (7 downto 0)
51.     );
52. end Component;
53.
54. Component BitwiseOperator is
55.     Port ( A : in STD_LOGIC_VECTOR (7 downto 0);
56.           B : in STD_LOGIC_VECTOR (7 downto 0);
57.           Type_Sel : in STD_LOGIC_VECTOR (1 downto 0);
58.           O : out STD_LOGIC_VECTOR (7 downto 0));
59. end Component;
60.

```

```

61. Signal OP ,Sum, Greater, Lower, Product, Remainder, Quotient, D, C, BO : STD_LOGIC_VECTOR (7
downto 0);
62. Signal Add : STD_LOGIC;
63.
64. begin
65.
66.
67. Add <= Not Sub;
68.
69. Div : Divider
70.     Port Map(
71.         A => A,
72.         B => B,
73.         S => Quotient,
74.         C => Remainder
75.     );
76.
77. Mult : Multiplier
78.     Port Map(
79.         A => A,
80.         B => B,
81.         S => Product
82.     );
83. Comp : Comparator
84.     Port Map(
85.         A => A,
86.         B => B,
87.         G => Greater,
88.         L => Lower
89.     );
90.
91.
92. addsub : c_addsub_0
93.     PORT MAP(
94.         A => A,
95.         B => B,
96.         ADD => Add,
97.         C_OUT => Overflow,
98.         S => Sum
99.     );
100.
101. BU : BitwiseOperator
102.     PORT MAP(
103.         A => A,
104.         B => B,
105.         Type_Sel => T,
106.         O => BO
107.     );
108.
109. Div_Sel : Process(T)
110. begin
111.     case T is
112.         when "00" => D <= Quotient;
113.         when "01" => D <= Remainder;
114.         when others => D <= (others => '0');
115.     end case;
116. end process;
117.
118. Comp_Sel : Process(T)
119. begin
120.     case T is
121.         when "00" => C <= Greater;
122.         when "01" => C <= Lower;
123.         when others => C <= (others => '0');
124.     end case;

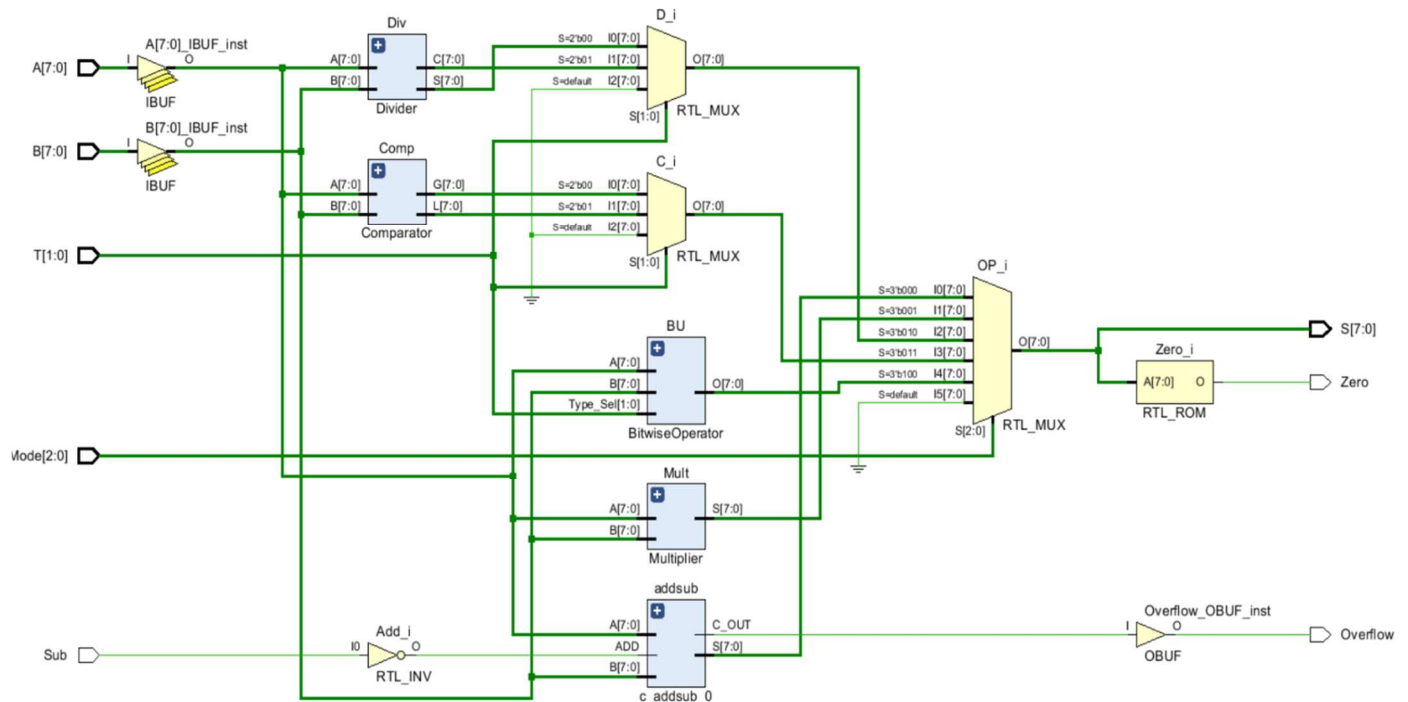
```

```

125. end process;
126.
127. OP_Sel : Process(Mode)
128. begin
129.     case Mode is
130.         when "000" => OP <= Sum;
131.         when "001" => OP <= Product;
132.         when "010" => OP <= D;
133.         when "011" => OP <= C;
134.         when "100" => OP <= B0;
135.         when others => OP <= (others => '0');
136.     end case;
137. end process;
138.
139. S <= OP;
140. Zero <= '1' when OP = "00000000" else '0';
141.
142. end Behavioral;

```

Design Schematic - Extended AU



Extended Program ROM

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity Program_ROM is
6.     port(
7.         ROM_address : in STD_LOGIC_VECTOR (2 downto 0);
8.         I            : out STD_LOGIC_VECTOR (16 downto 0);
9.         Program      : in STD_LOGIC_VECTOR (1 downto 0)
10.    );
11. end Program_ROM;
12.
13. architecture Behavioral of Program_ROM is
14.
15.     type ROM_Type is array (0 to 7) of STD_LOGIC_VECTOR(16 downto 0);
16.
17.     constant Program1 : ROM_Type := (
18.         "01011100000000111", -- MOVI R7, 7
19.         "01000100000000001", -- MOVI R1, 1
20.         "00100000100000000", -- SUB  R0, R1
21.         "01001000000000111", -- MOVI R2, 8
22.         "00001000100000000", -- ADD  R2, R1
23.         "00011101000000000", -- ADD  R7, R2
24.         "01101000000000110", -- JZR  R2, 6
25.         "01100000000000100" -- JZR  R0, 4
26.    );
27.
28.     constant Program2 : ROM_Type := (
29.         "01011100000000111", -- MOVI R7, 7
30.         "01011000000000010", -- MOVI R6, 2
31.         "10011111000000000", -- MUL  R7, R6
32.         "01010100000000100", -- MOVI R5, 4
33.         "10111110100000000", -- DIV  R7, R5, Q
34.         "01011000000011011", -- MOVI R6, 27
35.         "10111010100000001", -- DIV  R6, R5, R
36.         "01100000000000111" -- HALT
37.    );
38.
39.     constant Program3 : ROM_Type := (
40.         "01011100000110010", -- MOVI R7, 50
41.         "01011000000000111", -- MOVI R6, 15
42.         "01010100000011110", -- MOVI R5, 30
43.         "01010000000000100", -- MOVI R4, 12
44.         "11011111000000000", -- COMP R7, R6, G
45.         "11010110000000001", -- COMP R5, R4, L
46.         "11100000000000110", -- DIV  R7, R5, R
47.         "01100000000000111" -- HALT
48.    );
49.
50.     constant Program4 : ROM_Type := (
51.         "01011000011111000", -- MOVI R6, 0xF8 (1111 1000) 248
52.         "01011100000000111", -- MOVI R7, 0x0F (0000 1111) 15
53.         "11111111000000000", -- BITWISE R7, R6, AND ; R6 := R7 AND R6 = 00001000 = 8
54.         "11111011100001000", -- MOVI R6, 0xF8 (0000 1000) 8
55.         "01011000001000000", -- MOVI R6, 0x40 (0100 0000) 64
56.         "11111111000000001", -- BITWISE R7, R6 OR 48(0100 1000) 72
57.         "01001100101000001", -- MOVI R3, 0x41 (0100 0001) 135
58.         "11101100000000001" -- BITWISE R3 NOT 0xbE (1011 1110) 110
59.    );
60.
61.

```

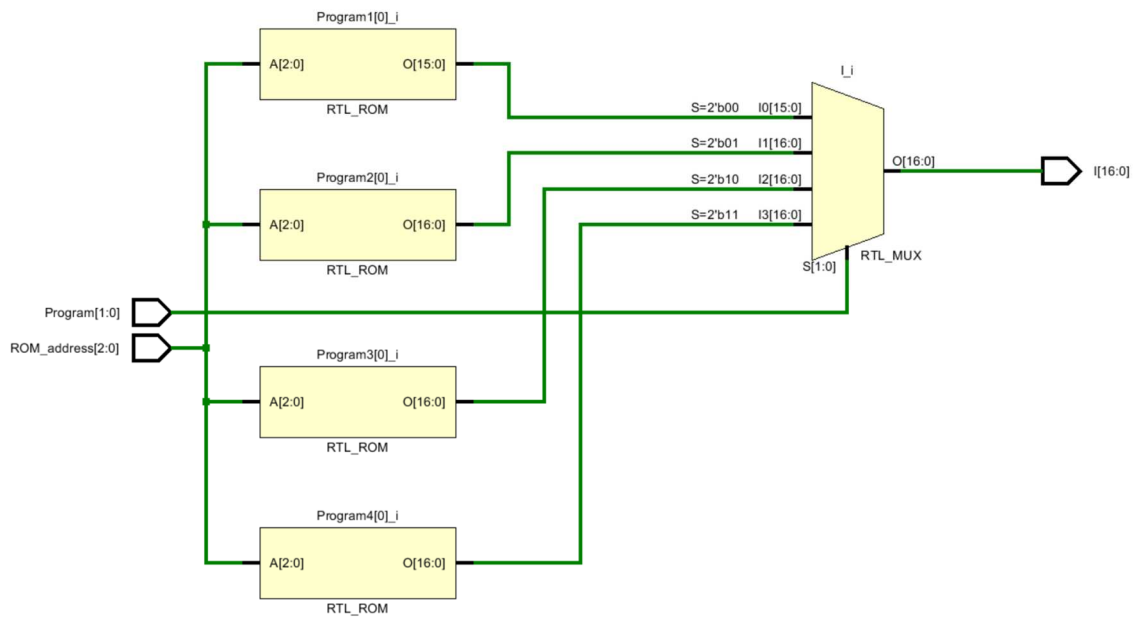


```

62. begin
63.
64.   process(ROM_address, Program)
65.     variable addr : integer;
66.     begin
67.       addr := to_integer(unsigned(ROM_address));
68.
69.       if addr < 8 then
70.         case Program is
71.           when "00" => I <= Program1(addr);
72.           when "01" => I <= Program2(addr);
73.           when "10" => I <= Program3(addr);
74.           when "11" => I <= Program4(addr);
75.           when others => I <= (others => '0');
76.         end case;
77.       else
78.         I <= (others => '0');
79.       end if;
80.     end process;
81.
82. end Behavioral.

```

Design Schematic - Extended Program ROM



Display Driver 1

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity Display_Driver is
6.     Port (
7.         Clk          : in  STD_LOGIC;
8.         Type_Sel     : in  STD_LOGIC;
9.         R0, R1, R2, R3, R4, R5, R6, R7 : in  STD_LOGIC_VECTOR (7 downto 0);
10.        Reg_Sel      : in  STD_LOGIC_VECTOR (2 downto 0);
11.        PC_in        : in  STD_LOGIC_VECTOR (2 downto 0);
12.        driver_signal: out  STD_LOGIC_VECTOR (6 downto 0);
13.        data          : out  STD_LOGIC_VECTOR (7 downto 0);
14.        Anode         : out  STD_LOGIC_VECTOR (3 downto 0)
15.    );
16. end Display_Driver;
17.
18. architecture Behavioral of Display_Driver is
19.     component MUX8 is
20.         Port (
21.             Sel : in  STD_LOGIC_VECTOR(2 downto 0);
22.             I0  : in  STD_LOGIC_VECTOR(7 downto 0);
23.             I1  : in  STD_LOGIC_VECTOR(7 downto 0);
24.             I2  : in  STD_LOGIC_VECTOR(7 downto 0);
25.             I3  : in  STD_LOGIC_VECTOR(7 downto 0);
26.             I4  : in  STD_LOGIC_VECTOR(7 downto 0);
27.             I5  : in  STD_LOGIC_VECTOR(7 downto 0);
28.             I6  : in  STD_LOGIC_VECTOR(7 downto 0);
29.             I7  : in  STD_LOGIC_VECTOR(7 downto 0);
30.             O   : out STD_LOGIC_VECTOR(7 downto 0)
31.         );
32.     end component;
33.
34.     component Slow_Clk_Bin is
35.         generic (
36.             N : INTEGER := 26
37.         );
38.         Port (
39.             Clk_in  : in  STD_LOGIC;
40.             Clk_out : out STD_LOGIC
41.         );
42.     end component;
43.
44.     signal D          : unsigned(1 downto 0) := (others => '0');
45.     signal Selected_Value : STD_LOGIC_VECTOR(7 downto 0);
46.     signal SC          : STD_LOGIC;
47.     type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
48.
49.     signal D0 : rom_type := (
50.         "1000000", -- 0
51.         "1111001", -- 1
52.         "0100100", -- 2
53.         "0110000", -- 3
54.         "0011001", -- 4
55.         "0010010", -- 5
56.         "0000010", -- 6
57.         "1111000", -- 7
58.         "0000000", -- 8
59.         "0010000", -- 9
60.         "0001000", -- A
61.         "0000011", -- B

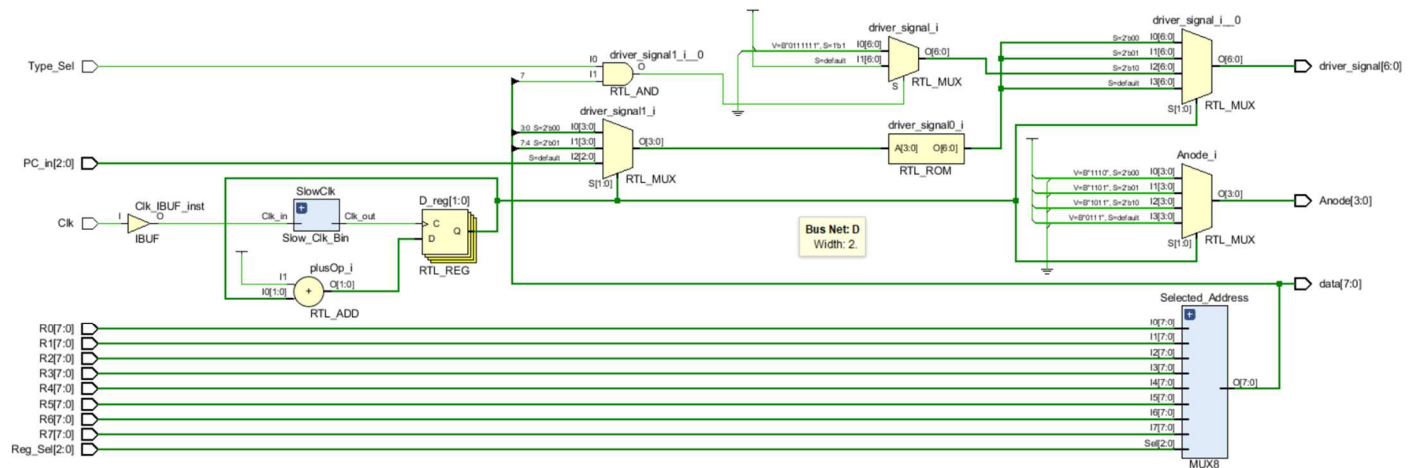
```

```

62.         "1000110", -- C
63.         "0100001", -- D
64.         "0000110", -- E
65.         "0001110" -- F
66.     );
67.
68. begin
69.     Selected_Address : MUX8
70.     Port Map (
71.         Sel => Reg_Sel,
72.         I0  => R0,
73.         I1  => R1,
74.         I2  => R2,
75.         I3  => R3,
76.         I4  => R4,
77.         I5  => R5,
78.         I6  => R6,
79.         I7  => R7,
80.         0   => Selected_Value
81.     );
82.
83.     SlowClk : Slow_Clk_Bin
84.     generic map ( N => 16 )
85.     port map (
86.         Clk_in  => Clk,
87.         Clk_out => SC
88.     );
89.
90.     Display : process(SC)
91.     begin
92.         if rising_edge(SC) then
93.             D <= D + 1;
94.         end if;
95.     end process;
96.
97.     Display_drive : process(D, Selected_Value, Type_Sel, PC_in)
98.     begin
99.         case D is
100.            when "00" =>
101.                driver_signal <= D0(to_integer(unsigned(Selected_Value(3 downto 0))));
102.                Anode <= "1110";
103.            when "01" =>
104.                driver_signal <= D0(to_integer(unsigned(Selected_Value(7 downto 4))));
105.                Anode <= "1101";
106.            when "10" =>
107.                if Type_Sel = '1' and Selected_Value(7) = '1' then
108.                    driver_signal <= "0111111";
109.                else
110.                    driver_signal <= "1111111";
111.                end if;
112.                Anode <= "1011";
113.            when others => -- "11"
114.                driver_signal <= D0(to_integer(unsigned(PC_in)));
115.                Anode <= "0111";
116.        end case;
117.    end process;
118.
119.    data <= Selected_Value;
120. end Behavioral;
121.

```

Design Schematic – Display Driver 1



Display Driver 2

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity Display_Driver_Nat is
6.     Port (
7.         Clk          : in  STD_LOGIC;
8.         Type_Sel     : in  STD_LOGIC;
9.         R0, R1, R2, R3, R4, R5, R6, R7 : in  STD_LOGIC_VECTOR (7 downto 0);
10.        Reg_Sel      : in  STD_LOGIC_VECTOR (2 downto 0);
11.        driver_signal: out STD_LOGIC_VECTOR (6 downto 0);
12.        Anode         : out STD_LOGIC_VECTOR (3 downto 0)
13.    );
14. end Display_Driver_Nat;
15.
16. architecture Behavioral of Display_Driver_Nat is
17.
18.     component MUX8 is
19.         Port (
20.             Sel : in  STD_LOGIC_VECTOR(2 downto 0);
21.             I0  : in  STD_LOGIC_VECTOR(7 downto 0);
22.             I1  : in  STD_LOGIC_VECTOR(7 downto 0);
23.             I2  : in  STD_LOGIC_VECTOR(7 downto 0);
24.             I3  : in  STD_LOGIC_VECTOR(7 downto 0);
25.             I4  : in  STD_LOGIC_VECTOR(7 downto 0);
26.             I5  : in  STD_LOGIC_VECTOR(7 downto 0);
27.             I6  : in  STD_LOGIC_VECTOR(7 downto 0);
28.             I7  : in  STD_LOGIC_VECTOR(7 downto 0);
29.             O   : out STD_LOGIC_VECTOR(7 downto 0)
30.         );
31.     end component;
32.
33.     component Slow_Clk_Bin is
34.         generic (
35.             N : INTEGER := 26
36.         );
37.         Port (
38.             Clk_in  : in  STD_LOGIC;
39.             Clk_out : out STD_LOGIC
40.         );
41.     end component;
42.
43.     signal D          : unsigned(1 downto 0) := (others => '0');
44.     signal Selected_Value : STD_LOGIC_VECTOR(7 downto 0);
45.     signal SC         : STD_LOGIC;
46.
47.     signal Dec_Val : integer range -128 to 255 := 0;
48.     signal Hundreds, Tens, Ones : integer range 0 to 9 := 0;
49.     signal Is_Negative : boolean := false;
50.
51.     type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
52.
53.     signal D0 : rom_type := (
54.         "1000000", -- 0
55.         "1111001", -- 1
56.         "0100100", -- 2
57.         "0110000", -- 3
58.         "0011001", -- 4
59.         "0010010", -- 5
60.         "0000010", -- 6
61.         "1111000", -- 7

```

```

62.         "0000000", -- 8
63.         "0010000", -- 9
64.         "0001000", -- A
65.         "0000011", -- B
66.         "1000110", -- C
67.         "0100001", -- D
68.         "0000110", -- E
69.         "0001110" -- F
70.     );
71.
72. begin
73.
74. Selected_Address : MUX8
75.     Port Map (
76.         Sel => Reg_Sel,
77.         I0  => R0,
78.         I1  => R1,
79.         I2  => R2,
80.         I3  => R3,
81.         I4  => R4,
82.         I5  => R5,
83.         I6  => R6,
84.         I7  => R7,
85.         0   => Selected_Value
86.     );
87.
88. SlowClk : Slow_Clk_Bin
89.     generic map ( N => 16 )
90.     port map (
91.         Clk_in  => Clk,
92.         Clk_out => SC
93.     );
94.
95. process(SC)
96. begin
97.     if rising_edge(SC) then
98.         D <= D + 1;
99.     end if;
100. end process;
101.
102.
103. process(Selected_Value, Type_Sel)
104.     variable unsigned_val : unsigned(7 downto 0);
105.     variable signed_val   : signed(7 downto 0);
106.     variable abs_val      : integer;
107.
108. begin
109.     unsigned_val := unsigned(Selected_Value);
110.     signed_val   := signed(Selected_Value);
111.
112.     if Type_Sel = '1' then
113.
114.         Dec_Val <= to_integer(signed_val);
115.         Is_Negative <= (signed_val < 0);
116.         abs_val := abs(to_integer(signed_val));
117.     else
118.         Dec_Val <= to_integer(unsigned_val);
119.         Is_Negative <= false;
120.         abs_val := to_integer(unsigned_val);
121.     end if;
122.
123.     Hundreds <= (abs_val / 100) mod 10;
124.     Tens      <= (abs_val / 10) mod 10;
125.     Ones      <= abs_val mod 10;
126. end process;

```

```
127.
128.
129. Display_drive : process(D, Hundreds, Tens, Ones, Is_Negative, Type_Sel)
130. begin
131.     case D is
132.         when "00" =>
133.             driver_signal <= D0(Ones);
134.             Anode <= "1110";
135.         when "01" =>
136.             driver_signal <= D0(Tens);
137.             Anode <= "1101";
138.         when "10" =>
139.             driver_signal <= D0(Hundreds);
140.             Anode <= "1011";
141.         when others =>
142.             if Type_Sel = '1' and Is_Negative then
143.                 driver_signal <= "0111111";
144.             else
145.                 driver_signal <= "1111111";
146.             end if;
147.             Anode <= "0111";
148.         end case;
149.     end process;
150.
151. end Behavioral;
152.
```

Conclusion

This nano processor design project was a comprehensive and collaborative effort that allowed us to apply the theoretical knowledge we've acquired in digital system design into a fully functional hardware implementation using VHDL. Over the course of the project, our team dedicated around **80 hours** in total, progressing through three major stages: the basic design, the optimized version, and an extended feature-rich processor.

Through this process, we deepened our understanding of how core digital components, such as adders, decoders, multiplexers, and registers are integrated to form a working processor. We also gained practical experience in VHDL coding, simulation, and synthesis, learning how design decisions affect performance and resource utilization, especially under constraints like LUT minimization.

In addition to technical growth, this project improved our skills in teamwork, time management, and collaborative debugging. It served as an excellent stepping stone in bridging theoretical digital logic with real world hardware design, preparing us for more advanced and complex engineering challenges ahead.